Regras de Negócio por Contexto Delimitado (Bounded Context)

pr_cadastramento_empresa_prov— Gestão de Contratos Empresariais PJ

> **Objetivo:** Consolidar todas as regras de negócio identificadas na procedure `humaster.pr_cadastramento_empresa_prov` (~5.000 linhas PL/SQL), organizadas pelos 18 Bounded Contexts mapeados na modelagem DDD.
>
> **Data:** 2026-03-11
> **Referências:** `ddd-modelagem-dominio.md`, `context-map-cadastramento-empresa.cml`, `BACKLOG-EPICO-FEATURES-USERSTORIES-PLSQL.md`, `ESTRATEGIA-REFATORACAO-PLSQL.md`

Sumário

BC	Nome	Subdomínio	Tipo	Qtd Regras
BC-01	Orquestração do Cadastramento	Efetivação de Contrato PJ	Core	12
BC-02	Validação de Proposta	Validação de Proposta	Supporting	33 checagens (21 Specs conceituais) + 27 critérios elegibilidade
BC-03	Gestão de Pessoa Jurídica	Cadastro de PJ	Supporting	7
BC-04	Endereço e Comunicação	Cadastro de PJ	Supporting	11
BC-05	Filial e área de Venda	Estrutura Comercial	Supporting	6
BC-06	Modelo de Negócio	Precificação e Modelo Comercial	Core	9
BC-07	Precificação	Precificação e Modelo Comercial	Core	8
BC-08	Empresa Conveniada	Efetivação de Contrato PJ	Core	18
BC-09	Coparticipação	Regulamentação e Participação	Core	12
BC-10	Carência e Compra de Carência	Regulamentação e Participação	Core	7
BC-11	Fidelização Contratual	Termos Contratuais	Supporting	5
BC-12	Acesso Internet / Portal	Acesso e Identidade	Generic	9
BC-13	Integração Odontológica	Integração com Odontologia	Supporting	6
BC-14	Auditoria e Log	Auditoria e Observabilidade	Generic	5
BC-15	Reembolso / Livre Escolha	Termos Contratuais	Supporting	5
BC-16	Mínimo Contratual e Breakeven	Termos Contratuais	Supporting	6
BC-17	Notificação por E-mail	Notificação	Generic	4
BC-18	Desconto e PIM	Estrutura Comercial	Supporting	3
			TOTAL	**152**

BC-01: Orquestração do Cadastramento

> **Subdomínio:** Efetivação de Contrato PJ (Core Domain)
> **Package alvo:** `pk_cadastramento_empresa`
> **Tabelas principais:** `tb_empresa_internet` (leitura), `tb_usuario_titular_internet`, `tb_usuario_dependente_internet`
> **Responsabilidade:** Coordenar todo o fluxo de criação de empresa conveniada.

Regras de Negócio

#	Código	Regra	Detalhamento
1	RN-01.01	**Somente propostas de inclusão são processadas**	A procedure só processa registros com `tp_operacao = '1'` (inclusão) da tabela `tb_empresa_internet`.
2	RN-01.02	**Fluxo transacional único**	Todo o processo de efetivação roda em uma única transação. O COMMIT só ocorre após todos os passos concluídos com sucesso.
3	RN-01.03	**Rollback em caso de erro**	Se qualquer etapa falhar, é feito ROLLBACK de toda a transação. A empresa parcialmente criada gera registro de pendência.
4	RN-01.04	**Registro de pendência em caso de falha**	Ao falhar, é inserido registro em `tb_pendencia_empresa_internet` com o erro. O COMMIT da pendência é feito separadamente (não participa do rollback).
5	RN-01.05	**Validação antes da geração de código**	No modelo DDD corrigido, as validações (BC-02) devem ser executadas **antes** da geração do código da empresa (`fn_empresa_conveniada`), evitando desperdício de códigos quando a validação falha. No legado, essa ordem está invertida (bug de design).
6	RN-01.06	**Substituição de código provisório por definitivo**	Após efetivação, os registros de `tb_usuario_titular_internet` e `tb_usuario_dependente_internet` que possuem código provisório (`'T' || nu_controle`) são atualizados para o código definitivo da empresa.
7	RN-01.07	**Execução de críticas pós-baixa**	Após a mudança de código provisório para definitivo, é executada `humaster.pr_con_emp_prov` para verificar consistência dos dados. > **CANDIDATA A REMOÇÃO:** Esta procedure possui alta probabilidade de ser removida do fluxo na refatoração. Motivos: (1) as críticas de saúde já são executadas **antes** da efetivação nas camadas 2 e 3 do BC-02 (`pr_critica_internet` + gate de críticas), tornando a reexecução pós-baixa **redundante**; (2) o código apresenta múltiplos `WHEN OTHERS THEN NULL` que silenciam erros; (3) usa `MAX(cd_log)+1` em vez de SEQUENCE para gerar PK (race condition); (4) faz múltiplos COMMITs parciais sem SAVEPOINT, gerando inconsistência em caso de falha; (5) implementa rollback lógico manual (re-UPDATE para reverter código provisório) em vez de usar transação atômica. O espelhamento odontológico (`pr_odon_cad_empresa_prov`) que ela dispara pode ser movido para a etapa de integrações pós-COMMIT (RN-01.08).
8	RN-01.08	**Integrações pós-COMMIT**	Os passos de notificação por e-mail (BC-17), desconto PIM (BC-18), cobrança falta param, espelhamento odonto (BC-13) e super simples (BC-13) são executados **após** o COMMIT. Falhas nesses passos não causam rollback do contrato.
9	RN-01.09	**Contexto compartilhado entre packages**	Um record `t_contexto_cadastro` (type PL/SQL) carrega as ~70 variáveis de contexto compartilhadas entre todos os packages (cd_empresa, cd_pessoa, cd_filial, cd_tabela, canal_venda, etc.).

#	Código	Regra	Detalhamento
10	RN-01.10	**Resultado de processamento**	Ao concluir com sucesso, `p_erro_controle` retorna `'procedimento efetuado para empresa,' || cd_empresa`. > **REFATORAR:** Este retorno por concatenação de string é frágil ? não possui tipagem, dificulta parsing pelo chamador e mistura dados (código da empresa) com mensagem textual. Na refatoração, substituir por um **record estruturado** (`t_resultado_processamento`) com campos separados: `fl_sucesso BOOLEAN`, `cd_empresa VARCHAR2`, `ds_mensagem VARCHAR2`, `cd_erro NUMBER`. Alternativamente, usar parâmetros `OUT` dedicados.
11	RN-01.11	**Três canais de entrada**	As propostas podem vir de: **TAFFIX** (Oracle Forms → digitação interna pelo canal de administradora de benefícios), **BITIX** (plataforma de vendas do corretor) ou **SIGO WebHap** (site do corretor SS ? portal web de vendas). Todos os canais gravam na mesma tabela `tb_empresa_internet`.
12	RN-01.12	**Processamento por nu_controle**	O parâmetro de entrada `p_nu_controle` identifica a proposta. Se informado, processa apenas essa proposta. O cursor `cr_empresa_internet` retorna propostas pendentes filtradas por esse controle.

BC-02: Validação de Proposta

> **Subdomínio:** Validação de Proposta (Supporting)
> **Package alvo:** `pk_validacao_proposta`
> **Tabelas de referência:** `tb_pessoa_lista_negra`, `tb_cep_logradouro`, `tb_cep_localidade`, `tb_uf`, `tb_tipo_logradouro`, `tb_natureza_juridica_wnce`, `tb_pendencia_empresa_internet`, `tb_usuario_critica_internet`, `tb_critica_liberada`, `tb_localidade_limitrofe`, `tb_empresa_agregado`, `tb_empresa_internet`
> **Responsabilidade:** Validação de campos e pré-requisitos de modelo **dentro** da `pr_cadastramento_empresa_prov`.
>
> **NOTA SOBRE ESCOPO:** Este documento concentra-se nas validações da `pr_cadastramento_empresa_prov`.
> As camadas 1, 2 e 3 (filtro de cursor, 13 pendências, gate de críticas) pertencem à `pr_efetiva_internet`
> e serão tratadas na **refatoração daquela rotina**. Estão documentadas aqui apenas como **contexto de pré-condição** para entender o estado da proposta ao chegar na `pr_cadastramento_empresa_prov`.

Visão Geral das Camadas de Validação

```
+-----------------------------------------------------------------------+
|  ESCOPO: pr_efetiva_internet (sera refatorada separadamente)          |
|                                                                       |
|  CAMADA 1 - Filtro do Cursor (selecao de propostas elegiveis)         |
|  CAMADA 2 - 13 Pendencias (pre-validacao antes da efetivacao)         |
|  CAMADA 3 - Gate de Criticas saude/odonto                             |
|                                                                       |
|  -> Pre-condicao: proposta chega na pr_cadastramento_empresa_prov     |
|     ja aprovada nestas 3 camadas.                                     |
+-----------------------------------------------------------------------+
                                    |
                                    v Proposta elegivel
                                    |
+-----------------------------------------------------------------------+
|  ESCOPO: pr_cadastramento_empresa_prov (este documento)               |
|                                                                       |
|  CAMADA 4 - 21 Specifications (validacao de campos)                   |
|  CAMADA 5 - Pre-requisitos de Modelo de Negocio (BC-06)               |
|                                                                       |
|  -> Falha -> raise_application_error + ROLLBACK + pendencia           |
```

```
|  -> OK -> prossegue para efetivacao (BC-03..BC-18)                  |
+---------------------------------------------------------------------+
```

Pré-condições (escopo `pr_efetiva_internet` — apenas referência)

> As seções abaixo descrevem as validações que **já foram executadas** pela `pr_efetiva_internet`
> antes de chamar `pr_cadastramento_empresa_prov`. Estão aqui apenas para **contexto de pré-condição**.
> A refatoração dessas validações será feita no escopo da `pr_efetiva_internet`.

▶ Camada 1 — Filtro do Cursor (clique para expandir)

Camada 1 — Filtro do Cursor (Seleção Inicial)

> **Escopo:** `pr_efetiva_internet` — será refatorada separadamente.

Critérios aplicados pelo cursor `cr_empresa_internet` na `pr_efetiva_internet` para selecionar propostas candidatas:

#	Critério	Coluna / Condição	Descrição
C1.01	**Operação de inclusão**	`tp_operacao = '1'`	Somente propostas de inclusão de nova empresa. Alterações e exclusões são ignoradas.
C1.02	**Status elegível**	`fl_status_processamento IN (0, 1, 8)`	**0** = digitado (primeira vez), **1** = pendente (reprocessamento), **8** = autorizado (aprovado manualmente). Status 2 (cancelado), 9 (processado) e 17 (divergência Neoway) são excluídos.
C1.03	**Vidas sinalizadas OK**	`fl_sinaliza_vidas_ok = 'S'`	Flag que indica que os beneficiários foram digitados/validados. Propostas sem beneficiários completos não são processadas.
C1.04	**Faixa PIM (1-29 vidas)**	`nu_total_empregado BETWEEN 1 AND 29`	Propostas PME (Pequena/Média Empresa) com 1 a 29 beneficiários. Acima de 29 vidas não entra no fluxo PIM automático.
C1.05	**Não é ex-cliente ativo**	`cd_empresa IS NULL` ou empresa com `fl_status != 2`	Se a proposta já tem `cd_empresa` preenchido com uma empresa ativa (`fl_status = 2`), não é reprocessada. Ex-clientes cancelados podem ser reativados.
C1.06	**Controle específico (quando informado)**	`nu_controle = p_nu_controle`	Se `p_nu_controle` é informado na `pr_cadastramento_empresa_prov`, processa apenas essa proposta. Se não informado, processa todas as elegíveis.

▶ Camada 2 — 13 Pendências do Orquestrador (clique para expandir)

Camada 2 — Pré-validação do Orquestrador (13 Pendências)

> **Escopo:** `pr_efetiva_internet` — será refatorada separadamente.

Validações executadas pela `pr_efetiva_internet` **antes** de chamar `pr_cadastramento_empresa_prov`. Se qualquer pendência for detectada, a proposta **não** é encaminhada para efetivação. Pendências são registradas em `tb_pendencia_empresa_internet`.

#	Pendência	Critério	Descrição	Spec DDD
C2.01	**Pend. 1** — Nome inválido	`nm_pessoa LIKE 'ASSOC%' OR 'CONDOM%' OR 'INSTITUTO%' OR 'SIND%'`	Razão social com prefixos de entidades que sugerem cadastro indevido (associações, condomínios, sindicatos, institutos).	SP-03

#	Pendência	Critério	Descrição	Spec DDD
C2.02	**Pend. 2** — Localidade fora da área	`nm_cidade NOT IN tb_localidade_limitrofe`	Cidade do contrato não pertence à área de abrangência da filial/operadora (localidade limítrofe).	SP-09
C2.03	**Pend. 3** — Vendedor × Filial divergente	Filial do vendedor/filial da localidade	O vendedor está vinculado a uma filial diferente da filial que atende a localidade do contrato. Exceção: vendedor nacional (`fl_vendedor_nacional = 'S'`).	SP-08
C2.04	**Pend. 4** — Dependente > 43 anos	`MONTHS_BETWEEN(sysdate, dt_nascimento)/12 > 43`	Dependente com mais de 43 anos sem parametrização de agregado vigente (`tb_empresa_agregado`).	SP-11
C2.05	**Pend. 5** — Sobrinho > 23 anos	`tp_dependente = 14 AND idade > 23`	Sobrinho (tipo de dependente 14) com mais de 23 anos excede o limite regulatório.	SP-12
C2.06	**Pend. 6** — Titular/Dependente >= 59 anos	`idade >= 59` (ou `>= 65` se `HABILITA_65ANOS = 'S'`)	Titular ou dependente com idade igual ou superior a 59 anos (ou 65, conforme feature flag `HABILITA_65ANOS`).	SP-10
C2.07	**Pend. 7** — CPF inválido/ausente	`fn_check_cic(nu_cpf) != 1` ou `nu_cpf IS NULL AND idade > 18`	CPF inválido (falha no dígito verificador) ou ausente para beneficiário maior de 18 anos.	SP-04
C2.08	**Pend. 8** — Qtd vidas fora da faixa	`COUNT(titulares + dependentes) NOT BETWEEN 1 AND 29`	Contagem real de vidas (titulares + dependentes ativos) diverge da faixa PIM (1-29).	SP-05
C2.09	**Pend. 10** — Inclusão tardia (> 6 dias)	`dt_digitacao - dt_cadastramento > 6`	Beneficiários digitados mais de 6 dias após o cadastramento da proposta (inclusão tardia).	SP-13
C2.10	**Pend. 11** — Crítica DIREX	`pr_critica_empresa_internet_1` retorna pendência	Crítica de diretoria (DIREX) → validação especial que bloqueia a efetivação até aprovação superior.	SP-14
C2.11	**Pend. 12** — Divergência Neoway	`fn_checa_divergencia(nu_controle) = 'S'`	Inconsistência detectada pelo serviço externo Neoway (validação cadastral/fiscal). **Exceção:** propostas BITIX (`cd_operados = 'BITIX'`) **não passam por essa checagem dentro da `pr_efetiva_internet`** (bloco protegido por `v_valida_emp_bitix = 0`, SACTI 1789837). **Atenção:** a exclusão aqui não significa ausência de Neoway para BITIX — a checagem é realizada por rotinas próprias do canal: `PR_VE_DIVERGENCIA_NEOWAY` (itera titulares/dependentes via `pr_verifica_divergencia`, processo 33, gravando `fl_status_processamento = '17'` em caso de divergência) e `fn_get_criticas_pendencias` (portão `COUNT_VALIDA`), ambas orquestradas **exclusivamente pelo `pk_venda_json`**.	SP-15
C2.12	**Pend. 13** — Titular >= 65 anos (limite estendido)	`idade >= 65 AND HABILITA_65ANOS = 'S'`	Quando a feature flag `HABILITA_65ANOS` está ativa, o limite sobe de 59 para 65 anos. Propostas com titulares ≥ 65 geram esta pendência específica.	SP-10
C2.13	**Pend. 9** — Crítica saúde/odonto	Críticas em `tb_usuario_critica_internet`	Existem críticas pendentes de beneficiários (titulares ou dependentes) que não foram liberadas em `tb_critica_liberada`. Só avaliada se `FL_CRITICA_SAUDE_ODONTO = 'S'`.	SP-14

▶ Camada 3 — Gate de Críticas (clique para expandir)

Camada 3 — Validação de Críticas (Gate de Controle)

> **Escopo:** `pr_efetiva_internet` — será refatorada separadamente.

Executada pela `pr_efetiva_internet` **após** as 13 pendências e **antes** de chamar `pr_cadastramento_empresa_prov`. Funciona como gate de controle adicional:

#	Critério	Condição	Descrição
C3.01	**Feature flag de crítica ativa**	`FL_CRITICA_SAUDE_ODONTO = 'S'` (via `fn_registro_sistema`)	Se a flag estiver inativa, esta camada é **ignorada** e a proposta vai direto para a efetivação.
C3.02	**Empresa ainda não existe**	`cd_empresa IS NULL`	A validação de críticas só é aplicada para empresas **novas**. Reativações de ex-clientes pulam esta etapa.
C3.03	**Crítica saúde**	`pr_critica_internet(nu_controle_saude)` sem críticas	A procedure de crítica valida dados de titulares e dependentes da proposta de saúde. Se existirem críticas, bloqueia.
C3.04	**Crítica odonto**	`pr_critica_internet(nu_controle_odonto)` sem críticas	Idem para a proposta odontológica vinculada (quando existir).
C3.05	**Sem críticas pendentes**	`COUNT(tb_usuario_critica_internet) = 0` (excluindo liberadas)	Nenhuma crítica ativa (não liberada em `tb_critica_liberada`) para os beneficiários da proposta.

Validações da `pr_cadastramento_empresa_prov` (escopo deste documento)

Camada 4 — 21 Specifications (Validação de Campos)

> **Escopo:** `pr_cadastramento_empresa_prov` — **este é o escopo principal deste documento.**
> Estas são as validações executadas **dentro** da procedure de efetivação, no bloco de validação (~linhas 250-960).

Camada 5 — Pré-requisitos de Modelo de Negócio (BC-06)

Critérios verificados durante a resolução do modelo de negócio, **dentro** da `pr_cadastramento_empresa_prov`:

#	Critério	Condição	Descrição
C5.01	**Modelo de negócio encontrado**	`cr_empresa_neg` ou `cr_empresa_neg_bitix` retorna registro	É obrigatório que exista um modelo de negócio para a combinação filial + natureza + faixa de empregados. Se não encontrado, a proposta é rejeitada (SP20).
C5.02	**Filial resolvida**	Cadeia vendedor -> area de venda -> filial resulta em `cd_filial` válido	Não é possível processar a proposta sem uma filial válida.
C5.03	**Coligada consistente (quando aplicável)**	`fn_valida_coligada_baixa` retorna OK	Se a empresa pertence a uma coligada, todas as empresas do grupo devem estar consistentes para prosseguir.

Diagrama de Fluxo das Camadas de Elegibilidade

```
+-------------------------------------------------------------------+
|  ESCOPO: pr_efetiva_internet (refatoracao separada)               |
|                                                                   |
|  +-------------------------------------------------------------+  |
|  |  CAMADA 1 - Filtro do Cursor                                |  |
|  |  tp_operacao=1, fl_status IN(0,1,8),                        |  |
|  |  fl_sinaliza_vidas_ok='S', empregados 1-29                  |  |
|  |  -> Nao elegivel -> proposta ignorada (nao processada)      |  |
|  +-------------------------------------------------------------+  |
|                         |                                         |
|                         v Elegivel                                |
```

```
  |  +-------------------------------------------------------------+   |
  |  |  CAMADA 2 - 13 Pendencias                                    |   |
  |  |  Nome invalido, localidade, vendedor/filial, idade,          |   |
  |  |  CPF, qtd vidas, inclusao tardia, DIREX, Neoway              |   |
  |  |  -> Pendencia -> tb_pendencia_empresa_internet + STOP        |   |
  |  +-------------------------------------------------------------+   |
  |                         |                                           |
  |                         v Zero pendencias                           |
  |  +-------------------------------------------------------------+   |
  |  |  CAMADA 3 - Gate de Criticas                                 |   |
  |  |  pr_critica_internet (saude + odonto)                        |   |
  |  |  So se FL_CRITICA_SAUDE_ODONTO='S' e cd_empresa IS NULL      |   |
  |  |  -> Criticas -> pendencia 9 + STOP                           |   |
  |  +-------------------------------------------------------------+   |
  |                         |                                           |
  |                         v Sem criticas                              |
  +-----------------------------------------------------------------------+
                            |
                            |
  +-----------------------------------------------------------------------+
  |  ESCOPO: pr_cadastramento_empresa_prov (este documento)             |
  |                                                                     |
  |  +-------------------------------------------------------------+   |
  |  |  >>> pr_cadastramento_empresa_prov(nu_controle) <<<          |   |
  |  +-------------------------------------------------------------+   |
  |                         |                                           |
  |  +-------------------------------------------------------------+   |
  |  |  CAMADA 4 - 21 Specifications (BC-02)                        |   |
  |  |  CNPJ, razao social, endereco, vendedor, natureza...         |   |
  |  |  -> Falha -> raise_application_error + ROLLBACK + pendencia  |   |
  |  +-------------------------------------------------------------+   |
  |                         |                                           |
  |                         v Todas as validacoes OK                    |
  |  +-------------------------------------------------------------+   |
  |  |  CAMADA 5 - Pre-requisitos de Modelo (BC-06)                 |   |
  |  |  Modelo de negocio, filial, coligada                         |   |
  |  |  -> Falha -> raise_application_error + ROLLBACK + pendencia  |   |
  |  +-------------------------------------------------------------+   |
  |                         |                                           |
  |                         v Modelo encontrado                         |
  |  +-------------------------------------------------------------+   |
  |  |  EFETIVACAO DO CONTRATO                                      |   |
  |  |  BC-03..BC-18 (PJ, Endereco, Preco, Copart, etc.)           |   |
  |  |  COMMIT -> Empresa Conveniada criada com sucesso             |   |
  |  +-------------------------------------------------------------+   |
  +-----------------------------------------------------------------------+
```

Observações sobre Sobreposição de Validações

> **Duplicação identificada:** As camadas 2 e 4 possuem **sobreposição parcial** de validações. Por exemplo, a validação de vendedor é feita tanto na pendência 3 (camada 2, na `pr_efetiva_internet`) quanto na Specification SP02 (camada 4, na `pr_cadastramento_empresa_prov`). Isso ocorre porque a `pr_efetiva_internet` foi criada como "pré-filtro" para evitar chamar a `pr_cadastramento_empresa_prov` desnecessariamente, mas as regras ficaram em

> dois lugares.
>
> **Decisão de escopo:**
>
> - As **camadas 1, 2 e 3** pertencem à `pr_efetiva_internet` e serão refatoradas **no escopo daquela rotina**.
> - As **camadas 4 e 5** pertencem à `pr_cadastramento_empresa_prov` e são o **escopo deste documento**.
> - A sobreposição será eliminada na refatoração: `pk_validacao_proposta` consolidará todas as validações, chamado de um único ponto. A duplicação entre camadas 2 e 4 será resolvida removendo as validações redundantes da `pr_efetiva_internet` (que passará a delegar para o Domain Service unificado).

Feature Flags que Alteram os Critérios

Flag	Valor Padrão	Efeito nos Critérios	Escopo
`MOVIMENTACAO_PIM_AUTOMATICO`	`'S'`	Se `'N'`, todo o bloco PIM automático (camadas 1-5) é desabilitado.	`pr_efetiva_internet`
`HABILITA_65ANOS`	`'N'`	Se `'S'`, o limite de idade das pendências 6/13 sobe de 59 para 65 anos.	`pr_efetiva_internet`
`FL_CRITICA_SAUDE_ODONTO`	`'S'`	Se `'N'`, a camada 3 (gate de críticas) é ignorada.	`pr_efetiva_internet`
`PENDENCIA_NEOWAY_PIM`	`'S'`	Se `'N'`, a pendência 12 (divergência Neoway) é ignorada na `pr_efetiva_internet`. **Não afeta o canal BITIX**, cujo fluxo Neoway é controlado pelos parâmetros `PK_VENDA_JSON_EXECUTA_NEOWAY` e `PR_VE_DIVERGENCIA_NEOWAY_FLAG` dentro do `pk_venda_json`.	`pr_efetiva_internet`
`FL_LOG_BAIXA_CONTROLE`	`'S'`	Se `'N'`, logs em `tb_log_baixa_controle` não são registrados.	`pr_cadastramento_empresa_prov`
`FL_ATIVA_PIM_ADM`	`'N'`	Se `'S'`, aplica desconto PIM via `pr_desconto_empresa`.	`pr_cadastramento_empresa_prov`

Diferenças por Canal de Origem

> As diferenças marcadas com **(E)** são de escopo `pr_efetiva_internet` (e, para o canal BITIX, `pk_venda_json` — que é o orquestrador equivalente para esse canal); as marcadas com **(C)** são de escopo `pr_cadastramento_empresa_prov`.

Critério	TAFFIX (Oracle Forms)	BITIX (Plataforma Corretor)	SIGO WebHap (Site Corretor SS)	Escopo
Fluxo de entrada	Digitação interna pelo canal de administradora de benefícios	Digitação pelo corretor na plataforma de vendas	Digitação pelo corretor no portal web	—
Tabela staging	`tb_empresa_internet`	`tb_empresa_internet`	`tb_empresa_internet`	—
Cursor modelo de negócio	`cr_empresa_neg` (SIGO)	`cr_empresa_neg_bitix` (BITIX)	`cr_empresa_neg` (SIGO)	(C)
Tabela coligada	`tb_empresa_coligada`	`tb_empresa_coligada_bitx`	`tb_empresa_coligada`	(C)
Override filial	`filial_modelo` via `tb_emp_internet_filial`	Não disponível	Não disponível	(C)

Critério	TAFFIX (Oracle Forms)	BITIX (Plataforma Corretor)	SIGO WebHap (Site Corretor SS)	Escopo
Checagem Neoway (Pend. 12)	Sim — via `fn_checa_divergencia` dentro da `pr_efetiva_internet`	**Não** na `pr_efetiva_internet` (excluído pelo bloco `v_valida_emp_bitix = 0`, SACTI 1789837). **Sim** via rotinas próprias do `pk_venda_json`: `PR_VE_DIVERGENCIA_NEOWAY` verifica CPF/nome de cada titular e dependente (processo 33); o portão `fn_get_criticas_pendencias` decide se `pr_cadastramento_empresa_prov` é acionada. Beneficiários com divergência recebem `fl_status_processamento = '17'`, podendo redirecionar a proposta para status 7 (fila manual) após o cadastramento.	Sim — via `fn_checa_divergencia` dentro da `pr_efetiva_internet`	(E)
Data início (ajuste)	Sem ajuste	Se `dt_inicio <= sysdate` -> `trunc(sysdate)`	Sem ajuste	(C)
Processamento coligada	`pr_processa_empresa_coligada`	`PR_COLIGA_EMPRESA_BITIX` + `PK_VENDA_JSON.pr_efetiva`	`pr_processa_empresa_coligada`	(C)
Identificador de origem	`cd_operados != 'BITIX'`	`cd_operados = 'BITIX'`	`cd_operados != 'BITIX'` (fluxo SIGO)	(C)

Regras de Negócio — Camada 4: Specifications (Validação de Campos)

> **Escopo:** `pr_cadastramento_empresa_prov` (~linhas 250-1320) — **validações que este documento detalha.**
> As Specifications são executadas em sequência **imperativa** (fail-fast). A primeira falha
> dispara `raise_application_error(-20201, ...)` e interrompe todo o processamento com ROLLBACK.
> Cada validação é precedida pelo padrão de log condicional odonto (BC-14).

Ordem Real de Execução no Código-Fonte

```
Linha ~248  -> SP02: Vendedor
Linha ~282  -> SP01a: CNPJ lista negra
Linha ~312  -> SP03: Filial (vendedor->area_venda->filial) + overrides
Linha ~380  -> SP04: Natureza empresa obrigatória + valores válidos
Linha ~420  -> SP05: Total empregados > 0
Linha ~440  -> SP20: Modelo de negócio (SIGO ou BITIX) — BC-06
Linha ~618  -> SP01b: CNPJ obrigatório
Linha ~632  -> SP01c: CNPJ/CPF dígito verificador (fn_check_cic)
Linha ~660  -> SP21: CAEPF válido (se CPF com fn_check_cic=1)
Linha ~690  -> SP19a: Razão social obrigatória
Linha ~710  -> SP19b: Razão social sem espaços duplos
Linha ~730  -> SP06a: Nome fantasia obrigatório
Linha ~750  -> SP06b: Nome fantasia sem espaços duplos
Linha ~770  -> SP07: Inscrição estadual sem espaços (se preenchida)
Linha ~800  -> SP08: CEP válido (cursor cr_logradouro)
Linha ~830  -> SP09a: UF obrigatória
Linha ~850  -> SP09b: UF cadastrada (cursor primary_cur -> tb_uf)
Linha ~880  -> SP09c: UF corresponde ao CEP (cr_logradouro2)
Linha ~920  -> SP10a: Cidade obrigatória
Linha ~940  -> SP10b: Cidade sem espaços duplos
Linha ~960  -> SP11: Bairro sem espaços duplos (se preenchido)
Linha ~985  -> SP12: Tipo logradouro cadastrado (tb_tipo_logradouro)
Linha ~1020 -> SP13a: Logradouro obrigatório
Linha ~1040 -> SP13b: Logradouro sem espaços duplos
```

```
Linha ~1070 -> SP14a: Complemento sem espaços duplos (se preenchido)
Linha ~1095 -> SP15a: E-mail sem espaços (se preenchido)
Linha ~1115 -> SP15b: E-mail contém '@'
Linha ~1135 -> SP15c: E-mail contém '.'
Linha ~1160 -> SP16: Dia pagamento entre 1 e 30
Linha ~1190 -> SP17: Validade contrato obrigatória
Linha ~1220 -> SP18: Vigência >= 6 meses (condicional)
Linha ~1260 -> SP19c: Contato obrigatório
Linha ~1280 -> SP19d: Contato sem espaços duplos
Linha ~1300 -> SP20: Cargo sem espaços duplos (se preenchido)
```

#	Código	Regra	Detalhamento Técnico	Exceção ao Falhar
1	RN-02.20 (SP02)	**Vendedor deve existir**	**1ª validação executada** (~linha 248). `SELECT 1 INTO l_aux FROM tb_vendedor_plano WHERE cd_vendedor_plano = st_e.cd_vendedor_plano`. Se `NO_DATA_FOUND` -> erro. > Verifica apenas **existência**, não verifica se está ativo (`fl_status`). Na `pr_efetiva_internet` (camada 2, pendência 3) já é validado vendedor→filial, mas aqui é apenas existência.	`raise_application_error(-20201, 'vendedor nao cadastrado ou nao ativo')`
2	RN-02.02 (SP01)	**CNPJ não pode estar na lista negra**	~linha 282. `SELECT COUNT(*) INTO l_aux FROM tb_pessoa_lista_negra WHERE (sysdate BETWEEN dt_inicial AND dt_final) AND nu_cpf_cnpj = st_e.nu_cgc_cpf`. Se `l_aux > 0` -> rejeita. Verifica vigência temporal da restrição (`dt_inicial`/`dt_final`). Exception `WHEN OTHERS THEN NULL` — se a query falhar, a validação é **ignorada** (falso negativo silencioso).	`raise_application_error(-20201, 'cnpj contem restricao no sistema e nao podera ser cadastrado')`
3	RN-02.21a (SP04)	**Natureza da empresa obrigatória**	~linha 380. `IF st_e.fl_natureza_empresa IS NULL THEN raise_application_error(...)`. Validação de nulidade simples.	`raise_application_error(-20201, 'obrigatorio informar a natureza da empresa')`
4	RN-02.21b (SP04)	**Natureza empresa em valores válidos**	~linha 395. `IF st_e.fl_natureza_empresa NOT IN (0,1,2,3,4,5,6,7,8,9) THEN raise_application_error(...)`. Valores hardcoded: 0=MEI, 1=Individual, 2=Ltda, 3=SA, 4=Cooperativa, 5=Filantrópica, 6=PME, 7=Associação, 8=Condomínio, 9=Simples. Na refatoração, usar enum `NaturezaEmpresa`.	`raise_application_error(-20201, 'escolha natureza empresa valida')`
5	RN-02.21c (SP05)	**Total de empregados > 0**	~linha 420. `IF nvl(st_e.nu_total_empregado, 0) = 0 THEN raise_application_error(...)`. Usa `NVL` para tratar NULL como 0. Não valida limite superior (já filtrado na camada 1 pela `pr_efetiva_internet`: 1-29).	`raise_application_error(-20201, 'necessario informar quantidade beneficiarios contratados.')`
6	RN-02.01a (SP01)	**CNPJ/CPF obrigatório**	~linha 618. `IF st_e.nu_cgc_cpf IS NULL THEN raise_application_error(...)`. Validação de nulidade. Executada **após** a resolução do modelo de negócio (camada 5), ou seja, o sistema já consumiu uma entrada no cursor de modelo antes de validar se o CNPJ existe. > Ordem invertida no legado → deveria ser antes do modelo.	`raise_application_error(-20201, 'obrigatorio informar cnpj')`
7	RN-02.01b (SP01)	**CNPJ/CPF dígito verificador válido**	~linha 632. `IF NOT pk_administracao.fn_check_cic(st_e.nu_cgc_cpf) IN (1, 2, 3) THEN raise_application_error(...)`. `fn_check_cic` retorna: **1** = CPF válido, **2** = CNPJ válido, **3** = CAEPF válido. Qualquer outro valor → dígito inválido. A function é do package `pk_administracao` (utilitário compartilhado).	`raise_application_error(-20201, 'digito de controle cnpj/cpf incorreto')`
8	RN-02.03 (SP21)	**CAEPF deve ser válido (quando CPF)**	~linha 660. `IF pk_administracao.fn_check_cic(st_e.nu_cgc_cpf) = 1 AND fn_check_caepf(nvl(st_e.nu_caepf, 0)) = 0 THEN raise_application_error(...)`. Só valida CAEPF quando o documento é CPF (retorno = 1). `fn_check_caepf` retorna 0 = inválido, 1 = válido. O CAEPF (Cadastro de Atividade Econômica da Pessoa Física) É obrigatório para MEI/produtor rural que usa CPF ao invés de CNPJ. Usado depois para resolver `cd_empresa_plano` via DECODE (~linha 1350).	`raise_application_error(-20201, 'numero CAEPF incorreto')`

#	Código	Regra	Detalhamento Técnico	Exceção ao Falhar
9	RN-02.04a (SP19)	**Razão social obrigatória**	~linha 690. `IF st_e.nm_pessoa_razao_social IS NULL THEN raise_application_error(...)`. Validação de nulidade.	`raise_application_error(-20201, 'obrigatorio informar a razao social')`
10	RN-02.04b (SP19)	**Razão social sem espaços duplos**	~linha 710. `IF instr(st_e.nm_pessoa_razao_social, ' ') > 0 THEN raise_application_error(...)`. Padrão de validação textual: `INSTR(campo, ' ') > 0` detecta dois espaços consecutivos. Usado em 8 campos diferentes. Na refatoração, centralizar em `fn_valida_texto_sem_espacos_duplos(p_valor, p_campo)`.	`raise_application_error(-20201, 'espacos duplos na razao social nao permitido')`
11	RN-02.05a	**Nome fantasia obrigatório**	~linha 730. `IF st_e.nm_fantasia IS NULL THEN raise_application_error(...)`. Obrigatoriedade incondicional — sempre exige nome fantasia.	`raise_application_error(-20201, 'obrigatorio informar um nome de fantasia')`
12	RN-02.05b	**Nome fantasia sem espaços duplos**	~linha 750. `IF instr(st_e.nm_fantasia, ' ') > 0 THEN raise_application_error(...)`. Mesmo padrão INSTR de espaço duplo.	`raise_application_error(-20201, 'espacos duplos no nome de fantasia nao permitido')`
13	RN-02.16	**Inscrição estadual sem espaços**	~linha 770. `IF st_e.nu_ident_insc_est IS NOT NULL THEN IF instr(st_e.nu_ident_insc_est, ' ') > 0 THEN raise_application_error(...)`. Validação condicional — só se preenchida. Diferente dos outros: verifica espaço **simples** (não duplo). Não valida formato/dígitos da IE.	`raise_application_error(-20201, 'espacos na inscricao estadual nao permitido')`
14	RN-02.08 (SP06)	**CEP deve ser válido**	~linha 800. Usa cursor `cr_logradouro(st_e.cd_cep_endereco, st_e.cd_uf_endereco, st_e.nm_cidade_endereco)`. Cursor definido como: `SELECT ... FROM tb_cep_logradouro WHERE cd_cep = p_cep UNION SELECT ... FROM tb_cep_localidade WHERE cd_cep = p_cep`. Busca em duas tabelas (logradouro específico ou localidade genérica). Se `%NOTFOUND` ? CEP inválido. Validação condicional: só executa se `cd_cep_endereco IS NOT NULL`.	`raise_application_error(-20201, 'cep nao e valido')`
15	RN-02.09a (SP07)	**UF obrigatória**	~linha 830. `IF st_e.cd_uf_endereco IS NULL THEN raise_application_error(...)`. Nulidade simples.	`raise_application_error(-20201, 'obrigatorio informa a uf')`
16	RN-02.09b (SP07)	**UF cadastrada no sistema**	~linha 850. Usa cursor `primary_cur(st_e.cd_uf_endereco)` definido como `SELECT cd_uf FROM tb_uf WHERE cd_uf = p_uf`. Se `%NOTFOUND` → UF não cadastrada.	`raise_application_error(-20201, 'unidade da federacao nao esta cadastrada')`
17	RN-02.09c (SP07)	**UF corresponde ao CEP**	~linha 880. Usa cursor `cr_logradouro2(st_e.cd_cep_endereco)` que retorna `cd_uf_logradouro` do CEP. `IF st_logradouro2.cd_uf_logradouro != st_e.cd_uf_endereco THEN raise_application_error(...)`. **Validação cruzada**: UF informada deve ser a mesma do CEP. Condicional a `cd_cep_endereco IS NOT NULL` e `cr_logradouro2%FOUND`.	`raise_application_error(-20201, 'uf nao corresponde ao uf do cep')`
18	RN-02.10a (SP08)	**Cidade obrigatória**	~linha 920. `IF st_e.nm_cidade_endereco IS NULL THEN raise_application_error(...)`. Nulidade simples.	`raise_application_error(-20201, 'obrigatorio informa a cidade')`
19	RN-02.10b (SP08)	**Cidade sem espaços duplos**	~linha 940. `IF instr(st_e.nm_cidade_endereco, ' ') > 0 THEN raise_application_error(...)`. Padrão INSTR espaço duplo.	`raise_application_error(-20201, 'espacos duplos nao permitido na cidade')`
20	RN-02.11 (SP09)	**Bairro sem espaços duplos**	~linha 960. `IF st_e.nm_bairro_endereco IS NOT NULL THEN IF instr(st_e.nm_bairro_endereco, ' ') > 0 THEN raise_application_error(...)`. Condicional — só valida se preenchido. > **Nao valida obrigatoriedade** do bairro, apenas formato. Na camada 2 (`pr_efetiva_internet`) o bairro pode ser validado como obrigatório.	`raise_application_error(-20201, 'espaco duplo no bairro nao permitido')`

#	Código	Regra	Detalhamento Técnico	Exceção ao Falhar
21	RN-02.12 (SP10)	**Tipo logradouro cadastrado**	~linha 985. `SELECT cd_tipo_logradouro INTO l_cd_tipo_logradouro FROM tb_tipo_logradouro WHERE cd_tipo_logradouro = st_e.cd_tipo_logradouro`. Se `NO_DATA_FOUND` ? tipo não cadastrado. Verifica existência na tabela de domínio `tb_tipo_logradouro` (RUA, AV, TRAV, AL, etc.).	`raise_application_error(-20201, 'tipo de logradouro nao esta cadastrado')`
22	RN-02.13a (SP11)	**Logradouro obrigatório**	~linha 1020. `IF st_e.nm_rua_endereco IS NULL THEN raise_application_error(...)`. Nulidade simples.	`raise_application_error(-20201, 'obrigatorio informar logradouro do endereco')`
23	RN-02.13b (SP11)	**Logradouro sem espaços duplos**	~linha 1040. `IF instr(st_e.nm_rua_endereco, ' ') > 0 THEN raise_application_error(...)`.	`raise_application_error(-20201, 'espacos duplos no logradouro do endereco nao permitido')`
24	RN-02.14 (SP12)	**Complemento sem espaços duplos**	~linha 1070. `IF st_e.ds_compl_endereco IS NOT NULL THEN IF instr(st_e.ds_compl_endereco, ' ') > 0 THEN raise_application_error(...)`. Condicional — só se preenchido. > **Nao valida tamanho máximo** (a coluna na tabela já limita).	`raise_application_error(-20201, 'espaco duplo no complemento endereco nao permitido')`
25	RN-02.15a (SP13)	**E-mail sem espaços**	~linha 1095. `IF st_e.ds_endereco_eletronico IS NOT NULL THEN IF instr(st_e.ds_endereco_eletronico, ' ') > 0 THEN raise_application_error(...)`. Diferente: verifica espaço **simples** (não duplo) — e-mails não podem ter nenhum espaço.	`raise_application_error(-20201, 'espacos no endereco eletronico nao permitido')`
26	RN-02.15b (SP13)	**E-mail contém '@'**	~linha 1115. `IF instr(st_e.ds_endereco_eletronico, '@') = 0 THEN raise_application_error(...)`. Validação mínima de formato: deve conter arroba.	`raise_application_error(-20201, 'endereco eletronico nao valido')`
27	RN-02.15c (SP13)	**E-mail contém '.'**	~linha 1135. `IF instr(st_e.ds_endereco_eletronico, '.') = 0 THEN raise_application_error(...)`. Validação mínima: deve conter ponto. > Validacao rudimentar ? não verifica posição do @ em relação ao ., domínio válido, etc. Na refatoração, usar regex ou `EmailValueObject.validate()`.	`raise_application_error(-20201, 'endereco eletronico nao valido')`
28	RN-02.17 (SP14)	**Dia de pagamento entre 1 e 30**	~linha 1160. `IF nvl(st_e.dt_dia_pagamento, 0) NOT BETWEEN 1 AND 30 THEN raise_application_error(...)`. Usa `NVL(..., 0)` para tratar NULL como 0 (fora da faixa). Dia 31 **não** é permitido. Usado depois em BC-12 para calcular `dia_limite_acesso` (5→10, outros→15).	`raise_application_error(-20201, 'dia pagamento dever estar na faixa 01-30')`
29	RN-02.18 (SP15)	**Validade do contrato obrigatória**	~linha 1190. `IF st_e.dt_validade_contrato IS NULL THEN raise_application_error(...)`. Nulidade simples. Não valida se data é futura ou passada.	`raise_application_error(-20201, 'obrigatorio informar data de validade do contrato')`
30	RN-02.19 (SP16)	**Vigencia >= 6 meses (condicional)**	~linha 1220. Lógica em 3 níveis: (1) `SELECT natureza_juridica INTO v_nat_juridica FROM tb_natureza_juridica_wnce WHERE nu_controle = p_nu_controle` ? busca natureza jurídica WNCE. Se `NO_DATA_FOUND` ? `v_nat_juridica := 1`. (2) `IF v_nat_juridica = 1 THEN` ? só aplica para natureza jurídica padrão (1). (3) `IF st_e.cd_modelo_negocio != 1 THEN` ? só aplica para modelos de negócio não-padrão (AFFIX e outros). (4) `IF st_e.dt_validade_contrato < add_months(trunc(sysdate, 'month'), 6) THEN raise_application_error(...)`. Calcula 6 meses a partir do **primeiro dia do mês atual** (`trunc(sysdate, 'month')`). SACTI 449802.	`raise_application_error(-20201, 'contrato com validade inferior a 6 meses')`
31	RN-02.06a (SP17)	**Contato obrigatório**	~linha 1260. `IF st_e.nm_contato_pessoa IS NULL THEN raise_application_error(...)`. Nulidade simples.	`raise_application_error(-20201, 'obrigatorio informar o nome do contato na empresa')`
32	RN-02.06b (SP17)	**Contato sem espaços duplos**	~linha 1280. `IF instr(st_e.nm_contato_pessoa, ' ') > 0 THEN raise_application_error(...)`.	`raise_application_error(-20201, 'espaco duplo no nome contato na empresa nao permitido')`

#	Código	Regra	Detalhamento Técnico	Exceção ao Falhar
33	RN-02.07 (SP18)	**Cargo sem espaços duplos**	~linha 1300. `IF st_e.nr_cargo_contato IS NOT NULL THEN IF instr(st_e.nr_cargo_contato, ' ') > 0 THEN raise_application_error(...)`. Condicional — só se preenchido. > **Nao valida obrigatoriedade** do cargo, apenas formato. O documento original indicava cargo como obrigatório mas o código não valida nulidade.	`raise_application_error(-20201, 'espaco duplo no nome do cargo nao permitido')`

Análise de Padrões da Camada 4

> **Total de validações reais no código:** 33 checagens (vs 21 Specifications conceituais agrupadas).
> As 21 Specifications do modelo DDD agrupam validações relacionadas (ex: SP07 = UF obrigatória + UF cadastrada + UF↔CEP = 3 checagens).

Padrão	Ocorrências	Impacto
Nulidade simples (`IS NULL`)	10	Podem ser agrupadas em um validator genérico `not_null(campo, mensagem)`
Espaço duplo (`INSTR(campo, ' ')`)	8	Centralizar em `fn_sem_espacos_duplos(campo, nome_campo)`
Existência em tabela de domínio (`SELECT INTO ... NO_DATA_FOUND`)	4	Vendedor, tipo logradouro, UF, CEP
Validação cruzada (campo vs campo)	2	UF↔CEP, CNPJ↔CAEPF
Validação de formato mínimo	3	E-mail (@, .), dia pagamento (range)
Validação condicional (só se preenchido)	6	IE, bairro, complemento, e-mail, cargo, CEP
Boilerplate de log por validação	~12 linhas	33 × 12 = **~396 linhas** só de log em validações

⚠ Issues Identificados na Camada 4

#	Issue	Impacto	Proposta
1	**Ordem invertida:** CNPJ é validado (~linha 618) **após** modelo de negócio (~linha 440)	Desperdiça processamento do modelo se CNPJ é inválido	Mover CNPJ para antes do modelo
2	**Lista negra silenciosa:** `EXCEPTION WHEN OTHERS THEN NULL` na query de lista negra	CNPJ bloqueado pode passar se a tabela estiver indisponível	Remover `WHEN OTHERS` ou logar
3	**Cargo não É obrigatório:** Código só valida formato, não nulidade	Inconsistência com requisito documentado	Adicionar `IF nr_cargo_contato IS NULL`
4	**Bairro não É obrigatório:** Código só valida formato, não nulidade	Possível bug — depende do requisito real	Verificar com PO
5	**E-mail validação rudimentar:** Apenas `@` e `.`	E-mails inválidos podem passar (`@.`, `a@b.`, etc.)	Usar regex ou VO `Email`
6	**396 linhas de boilerplate:** 33 blocos de log → 12 linhas cada	36% do código de validação → log repetitivo	Centralizar em `fn_registra_e_rejeita(msg)`

BC-03: Gestão de Pessoa Jurídica

> **Subdomínio:** Cadastro de Pessoa Jurídica (Supporting)
> **Package alvo:** `pk_pessoa_juridica`
> **Tabelas:** `tb_pessoa`, `tb_natureza_juridica_emp`
> **Responsabilidade:** CRUD da entidade Pessoa vinculada à empresa.

Regras de Negócio

#	Código	Regra	Detalhamento
1	RN-03.01	**Verificação de existência por CNPJ**	Antes de criar, verifica se já existe pessoa com o CNPJ (`nu_cgc_cpf`) e `cd_empresa_plano` informados na `tb_pessoa`. **SQL:** `SELECT cd_pessoa FROM tb_pessoa WHERE nu_cgc_cpf = :cnpj AND cd_empresa_plano = :plano`. Se `NO_DATA_FOUND`, `lcd_pessoa := null` e segue para INSERT. Se encontrar, reutiliza o `cd_pessoa` existente e segue para UPDATE. **Ref:** linhas ~1420-1430 do legado.
2	RN-03.02	**Geração de cd_pessoa para nova pessoa**	Novo código gerado em duas etapas: (1) `SELECT sq_pessoa.nextval INTO lcd_pessoa FROM dual`; (2) `lcd_pessoa := pk_administracao.fn_digito_out(lcd_pessoa)` ? a função `fn_digito_out` calcula e concatena o dígito verificador ao código sequencial, garantindo unicidade com check-digit. **Ref:** linhas ~1432-1434 do legado.
3	RN-03.03	**Determinação do tipo de pessoa**	`fl_tipo_pessoa` é determinado por `DECODE(pk_administracao.fn_check_cic(nu_cgc_cpf), 1, 1, 2)`: retorno `1` do `fn_check_cic` = CPF → `fl_tipo_pessoa = 1` (Física); retorno `2` = CNPJ ? `fl_tipo_pessoa = 2` (Jurídica); retorno `3` = CAEPF ? `fl_tipo_pessoa = 2`. **Impacto:** Determina se o contrato é PJ (CNPJ padrão) ou PF equiparado (CAEPF). **Ref:** linhas ~1442, 1478 do legado.
4	RN-03.04	**Identificação única PJ**	A PK de `tb_pessoa` é a combinação `cd_pessoa` + `cd_empresa_plano`. O `cd_empresa_plano` ? resolvido em BC-05 (RN-05.04) pela cadeia vendedor -> area de venda -> filial, com fallback para `1` (Hapvida padrão) ou `14` (NDI SP para BITIX). Isso permite que o mesmo CNPJ exista em operadoras diferentes (ex: Hapvida `cd_empresa_plano=1` e RN Saúde `cd_empresa_plano=7`).
5	RN-03.05	**Pessoa jurídica → pré-requisito para empresa conveniada**	O `lcd_pessoa` é usado como FK obrigatória no INSERT de `tb_empresa_conveniada.cd_pessoa` (BC-08, RN-08.02). Se o INSERT/UPDATE de `tb_pessoa` falhar, o `raise_application_error(-20201)` interrompe o fluxo e a empresa conveniada não é criada. **Dependência:** BC-03 -> BC-08 (Customer-Supplier).
6	RN-03.06	**Atualização de dados cadastrais**	Se a PJ já existir, executa UPDATE: `UPDATE tb_pessoa SET nm_pessoa_razao_social = :razao, nm_fantasia = :fantasia, fl_tipo_pessoa = DECODE(...), nu_ident_insc_est = :insc WHERE cd_pessoa = :pessoa`. Não atualiza `nu_cgc_cpf` (imutável) nem `cd_empresa_plano` (parte da PK). **Ref:** linhas ~1473-1483 do legado.
7	RN-03.07	**Atualização de natureza jurídica**	A natureza jurídica é buscada de `tb_natureza_juridica_wnce WHERE nu_controle = p_nu_controle` e armazenada em `v_nat_juridica`. Valor padrão `1` se `NO_DATA_FOUND`. Este valor é usado em RN-02.19 para determinar se a vigência mínima de 6 meses deve ser validada (`v_nat_juridica = 1` e `cd_modelo_negocio != 1`). **Ref:** linhas ~1200-1206 do legado.

BC-04: Endereço e Comunicação

> **Subdomínio:** Cadastro de Pessoa Jurídica (Supporting)
> **Package alvo:** `pk_endereco_comunicacao`
> **Tabelas:** `tb_endereco_pessoa`, `tb_empresa_endereco`, `tb_contato_pessoa`, `tb_meio_comunicacao_pessoa`
> **Responsabilidade:** Gerenciar endereços, contatos e meios de comunicação.

Regras de Negócio

#	Código	Regra	Detalhamento
1	RN-04.01	**Cadastro de endereço de correspondência**	INSERT em `tb_endereco_pessoa` com os campos da proposta: `cd_pessoa`, `nu_endereco`, `cd_cep_endereco`, `nm_cidade_endereco`, `cd_uf_endereco`, `cd_tipo_logradouro`, `nm_rua_endereco`, `nm_bairro_endereco`, `ds_compl_endereco`, `fl_tipo_endereco = 2` (comercial), `ds_endereco_eletronico`. O `cd_endereco_correspondencia` é calculado por `MAX(cd_endereco_correspondencia) + 1` dentro da mesma pessoa. **Ref:** linhas ~3250-3280 do legado.
2	RN-04.02	**Tratamento especial AFFIX para endereço**	Quando `fn_checa_contrato_affix(p_nu_controle, 'S') = 'S'`, o endereço NÃO é criado. Em vez disso, reutiliza o endereço existente: `SELECT MAX(cd_endereco_correspondencia) INTO lcd_endereco_correspondencia FROM tb_endereco_pessoa WHERE cd_pessoa = lcd_pessoa`. Depois, insere apenas o vínculo em `tb_empresa_endereco`. A lógica é controlada por IF/ELSE no nível do bloco de endereço. **Ref:** linhas ~3210-3240 (AFFIX) vs ~3250-3310 (normal).
3	RN-04.03	**Vinculação do endereço à empresa**	INSERT em `tb_empresa_endereco(cd_empresa_conveniada, cd_pessoa, cd_endereco_correspondencia, cd_tipo_empresa_endereco)` com `cd_tipo_empresa_endereco = 1` (fatura). Em caso de `DUP_VAL_ON_INDEX`, faz UPDATE do `cd_endereco_correspondencia`. **Ref:** linhas ~3230 (AFFIX) e ~3290 (normal) do legado.

#	Código	Regra	Detalhamento
4	RN-04.04	**Cadastro de contato**	INSERT/UPDATE em `tb_contato_pessoa(cd_pessoa, nm_contato_pessoa, nr_cargo_contato, dt_nascimento)` com os dados do contato da proposta (`st_e.nm_contato_pessoa`, `st_e.nr_cargo_contato`, `st_e.dt_nascimento_contato`). O contato é identificado pela combinação `cd_pessoa` + `nm_contato_pessoa`. **Ref:** linhas ~3370-3400 do legado.
5	RN-04.05	**Cadastro de meios de comunicação por tipo**	Para cada campo de telefone não-nulo da proposta, é feito INSERT em `tb_meio_comunicacao_pessoa(cd_pessoa, cd_tipo_meio_comunicacao, cd_ordem_meio_comunicacao, nu_telefone)`. Mapeamento fixo: `nu_telefone` ? tipo `1`; `nu_fax` ? tipo `3`; `nu_telex` ? tipo `4`; `nu_movel` ? tipos `5` e `8` (dois registros); `nu_bip` ? tipo `6`; `nu_cx_postal` ? tipo `7`; `ds_endereco_eletronico` ? tipo `9`. **Ref:** linhas ~3410-3520 do legado.
6	RN-04.06	**Celular gera dois registros**	O campo `st_e.nu_movel` (celular) gera **dois** INSERTs em `tb_meio_comunicacao_pessoa`: um com `cd_tipo_meio_comunicacao = 5` e outro com `cd_tipo_meio_comunicacao = 8`, ambos com o mesmo número. A `cd_ordem_meio_comunicacao` é incrementada separadamente para cada registro. Isso provavelmente é legado de quando existiam dois tipos de celular (ex: comercial e pessoal).
7	RN-04.07	**Validação de CEP contra tabelas de referência**	O CEP é validado pelo cursor `cr_logradouro` que faz UNION de `tb_cep_logradouro` (CEPs de logradouro) e `tb_cep_localidade` (CEPs genéricos de cidade). Se `%NOTFOUND` -> `raise_application_error(-20201, 'cep não ? valido')`. Adicionalmente, `cr_logradouro2` verifica se a UF do CEP coincide com a UF informada ? divergência gera erro separado. **Ref:** linhas ~860-930, 930-960 do legado.
8	RN-04.08	**Ordem de meio de comunicação**	A variável `lcd_ordem_meio_comunicacao` é incrementada (`+1`) a cada INSERT em `tb_meio_comunicacao_pessoa`, garantindo sequência dentro do cadastro da pessoa. é inicializada buscando `NVL(MAX(cd_ordem_meio_comunicacao), 0)` para a pessoa.
9	RN-04.09	**E-mail como meio de comunicação tipo 9**	O campo `ds_endereco_eletronico` é validado antes: não pode conter espaços (`INSTR(:email, ' ') > 0` ? erro), deve conter `@` e `.`. Se válido, é inserido como tipo `9` em `tb_meio_comunicacao_pessoa`. **Validações:** (1) sem espaços; (2) contém `@`; (3) contém `.`. **Ref:** linhas ~1095-1145 (validação) e ~3500-3520 (insert).
10	RN-04.10	**Apenas meios não-nulos são cadastrados**	Cada bloco de INSERT de meio de comunicação é protegido por `IF st_e.nu_telefone IS NOT NULL THEN ... END IF`. Campos nulos são simplesmente ignorados sem gerar erro. Isso resulta em 0 a 9 registros por empresa dependendo dos campos preenchidos na proposta.
11	RN-04.11	**Registro de e-mail corporativo**	Após o COMMIT, se `st_e.ds_endereco_eletronico IS NOT NULL`, é feito INSERT em `tb_acesso_dados_empresa(cd_empresa_conveniada, ds_email_adm, cd_pessoa, cd_acesso, cd_tipo_acesso, dt_cadastro, cd_operador)` com `cd_tipo_acesso = 5`. Este INSERT possui `EXCEPTION WHEN OTHERS THEN NULL` ? falha é silenciada. **Ref:** linhas ~4672-4695 do legado. Sacti 1941776 (Roberto Santos).

BC-05: Filial e área de Venda

> **Subdomínio:** Estrutura Comercial (Supporting)
> **Package alvo:** `pk_filial_area_venda`
> **Tabelas:** `tb_vendedor_plano`, `tb_area_venda`, `tb_filial`, `tb_emp_internet_filial`
> **Responsabilidade:** Resolver a filial correta para o contrato.

Regras de Negócio

#	Código	Regra	Detalhamento
1	RN-05.01	**Resolução de filial via cadeia vendedor -> area de venda -> filial**	**SQL:** `SELECT a.cd_filial FROM tb_vendedor_plano v, tb_area_venda a WHERE cd_vendedor_plano = :vendedor AND a.cd_area_venda = v.cd_area_venda`. Se `WHEN OTHERS` -> `raise_application_error(-20201, 'impossivel determinar a filial do vendedor')`. A cadeia é: `cd_vendedor_plano` (proposta) ? `tb_vendedor_plano.cd_area_venda` ? `tb_area_venda.cd_filial` ? `lcd_filial`. **Ref:** linhas ~328-350 do legado. **DS02:** `ResolucaoFilialService`.

#	Código	Regra	Detalhamento
2	RN-05.02	**Override por cd_filial_contrato**	Se `st_e.cd_filial_contrato IS NOT NULL`, a filial é sobrescrita: `lcd_filial := st_e.cd_filial_contrato`. Este campo é preenchido diretamente na proposta e tem precedência sobre a cadeia vendedor→filial. **Comentário no legado:** `-- Sacti 1026849`. **Ref:** linha ~353 do legado.
3	RN-05.03	**Override por filial_modelo (TAFFIX)**	**SQL:** `SELECT cd_filial_modelo FROM tb_emp_internet_filial WHERE nu_controle = :controle`. Se encontrar registro (canal TAFFIX), `lcd_filial := lcd_filial_modelo`. O `WHEN OTHERS` define `lcd_filial_modelo := null` (silencia erro, fallback para filial anterior). **Prioridade:** vendedor→filial < `cd_filial_contrato` < `filial_modelo`. **Ref:** linhas ~356-367 do legado.
4	RN-05.04	**Recuperação de cd_empresa_plano da filial**	**SQL:** `SELECT l.cd_empresa_plano FROM tb_filial l WHERE l.cd_filial = lcd_filial`. Define `l_cd_empresa_plano_v` que identifica a operadora: `1` = Hapvida, `7` = RN Saúde, `14` = NDI SP. **Fallback:** `WHEN OTHERS ? 0`. O `cd_empresa_plano` é usado em toda a resolução posterior: CAEPF, tb_pessoa, e-mail, modelo de negócio. **Ref:** linhas ~370-376 do legado.
5	RN-05.05	**Filial deve ser válida e ativa**	A filial resolvida deve existir em `tb_filial` para o `cd_empresa_plano`. A validação é implícita: se a filial não existe, a query da RN-05.04 retorna `WHEN OTHERS ? 0`, o que causa falha downstream na resolução do modelo de negócio (SP20). **Specification:** SP03 (`FilialValidaSpec`).
6	RN-05.06	**área de venda determina concessionária**	A `tb_area_venda` representa a concessionária/escritório de vendas. A resolução de `cd_empresa_plano` também pode vir da área de venda diretamente, via query UNION na resolução CAEPF: `SELECT f.cd_empresa_plano FROM tb_area_venda a, tb_filial f WHERE f.cd_filial = a.cd_filial AND a.cd_area_venda = st_e.cd_area_venda`. **Ref:** linhas ~1330-1370 (query UNION com 4 fontes para `cd_empresa_plano`).

BC-06: Modelo de Negócio (Parametrização Comercial)

> **Subdomínio:** Precificação e Modelo Comercial (Core Domain)
> **Package alvo:** `pk_modelo_negocio`
> **Tabelas:** `tb_empresa_neg`, `tb_empresa_neg_bitix`, `tb_empresa_neg_tabela`, `tb_empresa_neg_desconto`, `tb_empresa_neg_franquia`, `tb_empresa_neg_fator`, `tb_empresa_neg_controle`, `tb_empresa_neg_carencia`, `tb_empresa_neg_grupo`, `tb_empresa_neg_modulo`, `tb_empresa_neg_fidelizacao`, `tb_empresa_neg_isenta_copart`, `tb_empresa_neg_pln_esp`, `tb_empresa_coligada`, `tb_empresa_coligada_bitx`
> **Responsabilidade:** Buscar parâmetros do modelo de negócio com base em filial, natureza e quantidade de beneficiários.

Regras de Negócio

#	Código	Regra	Detalhamento
1	RN-06.01	**Resolução de modelo por filial + natureza + faixa de empregados**	**Cursor:** `cr_empresa_neg(cd_modelo_negocio, cd_filial, fl_natureza_empresa, nu_total_empregado, dt_inicio)` com filtros: `cd_modelo_negocio = NVL(:modelo, 1)`, `fl_natureza_empresa IN (:natureza, 'T')` (específico ou "Todos"), `cd_filial IN (:filial, '999')` (específico ou "Todas"), `nu_total_empregado BETWEEN qt_inicial AND qt_final`, `dt_inicio >= NVL(dt_referencia, sysdate)`. Usa subselect `MAX(nu_controle)` para pegar o registro mais recente. `ORDER BY dt_referencia DESC, fl_natureza_empresa, cd_filial` ? prioriza: mais recente, mais específico. **Ref:** linhas ~95-115 do legado.
2	RN-06.02	**Dois caminhos paralelos: SIGO e BITIX**	A bifurcação é controlada por `V_COUNT_BITIX`: `SELECT COUNT(1) FROM tb_empresa_internet WHERE cd_operados = 'BITIX' AND nu_controle = :controle`. Se `V_COUNT_BITIX = 0` ? fluxo SIGO (cursor `cr_empresa_neg`, tabela `tb_empresa_neg`); se `> 0` ? fluxo BITIX (cursor `cr_empresa_neg_bitix`, tabela `tb_empresa_neg`). > **Ambos os cursors são idênticos** (mesma query) ? a diferença está na passagem de parâmetros (BITIX usa `st_e.dt_assinatura` em vez de `st_e.dt_inicio`) e na resolução de coligada. **Candidato a unificação via Strategy Pattern. Ref:** linhas ~440-600 do legado.
3	RN-06.03	**Resolução de coligada (grupo empresarial)**	**SIGO:** `SELECT SUM(c.nu_total_empregado) FROM tb_empresa_coligada b, tb_empresa_internet c WHERE b.contrato = 'T' || c.nu_controle AND b.cod_contrato_mae = 'T' || :cod_mae`. O `cod_mae_sigo` ? obtido por `SELECT SUBSTR(d.cod_contrato_mae, 2) FROM tb_empresa_coligada d WHERE SUBSTR(d.contrato, 2) = :controle`. **BITIX:** `SELECT SUM(nu_total_empregado) FROM tb_empresa_coligada_bitx d, tb_empresa_internet e WHERE e.nu_controle = d.tprovisorio AND d.cd_proposta_mae = :proposta_mae`. Se o total da coligada > 0, substitui `nu_total_empregado` individual pelo total do grupo. **Ref:** linhas ~430-470 (SIGO) e ~540-570 (BITIX).

#	Código	Regra	Detalhamento
4	RN-06.04	**Modelo de negócio É obrigatório**	Se `cr_empresa_neg%NOTFOUND` (SIGO) ou `cr_empresa_neg_bitix%NOTFOUND` (BITIX) -> `raise_application_error(-20201, 'nao encontrato parametros para natureza/filial/nu beneficiarios')`. **Specification:** SP20 (`ModeloNegocioExistenteSpec`). O erro ? registrado em `tb_log_baixa_controle` com `fl_status = '15'` antes do raise. **Ref:** linhas ~500-520 e ~590-610 do legado.
5	RN-06.05	**Carga completa de tabelas filhas**	O `lnu_controle` (obtido do cursor `st_empresa_neg.nu_controle`) é o pivô para todas as queries subsequentes de staging: `tb_empresa_neg_tabela WHERE nu_controle = lnu_controle`, `tb_empresa_neg_desconto WHERE nu_controle = lnu_controle`, `tb_empresa_neg_franquia WHERE nu_controle = lnu_controle`, `tb_empresa_neg_fator`, `tb_empresa_neg_controle`, `tb_empresa_neg_carencia`, `tb_empresa_neg_grupo`, `tb_empresa_neg_modulo`, `tb_empresa_neg_fidelizacao`, `tb_empresa_neg_isenta_copart`, `tb_empresa_neg_pln_esp`, `tb_empresa_neg_isnt_cop_tp_ben`, `tb_empresa_cop_intern_pj`. **~15 tabelas filhas** carregadas por FOR cursor.
6	RN-06.06	**Determinação do canal de venda**	**Cálculo:** `IF nu_total_empregado BETWEEN 1 AND 29 THEN canal := 1` (varejo PIM); `ELSIF BETWEEN 30 AND 99 THEN canal := 2` (Middle Market); `ELSE canal := NULL` (grandes contas). O cálculo é repetido 3 vezes no código: para empregados individuais (linhas ~1875-1880), para BITIX coligada (linhas ~1885-1895), e para SIGO coligada (linhas ~1897-1905). **Candidato a centralização.**
7	RN-06.07	**Override de canal para BITIX/SIGO coligada**	Se `V_NU_TOTAL_BITIX > 0` (BITIX) ou `NU_TOTAL_EMPREGADO_SIGO > 0` (SIGO), o canal de venda é recalculado usando o total somado do grupo. Exemplo: empresa individual com 10 vidas (canal 1) pertencente a grupo com 50 vidas totais ? canal recalculado para 2 (Middle). **Ref:** linhas ~1885-1910 do legado.
8	RN-06.08	**Determinação de flag tabela geral**	**Condição:** `IF lcd_modelo_negocio != 1 AND lcd_tabela_saude IS NOT NULL THEN wfl_tabela_geral := 'S'`. Modelo `1` = padrão; qualquer outro modelo (ex: AFFIX, modelos 2-N) com tabela de saúde definida ? tabela geral compartilhada. Quando `wfl_tabela_geral = 'S'`, a tabela de preço do modelo é reutilizada (`lcd_tabela := lcd_tabela_saude`) em vez de criar nova ? evita N tabelas para a mesma filial/administradora. **Sacti:** 355061 (David Brandão). **Ref:** linhas ~525-535 do legado.
9	RN-06.09	**Índice de modelo de cálculo**	As variáveis `v_id_indice_empresa` e `v_cd_indice_modelo_calculo` são inicializadas em `0` e atualizadas a partir de `tb_indice_modelo_calculo_neg` via `tb_empresa_neg_controle`. O Índice de modelo de cálculo determina a regra de reajuste anual do contrato. é inserido em `tb_modelo_reaj_empresa(cd_modelo_reaj_empresa, cd_indice_modelo_calculo)` vinculado ? empresa. **Ref:** linhas ~2660-2690 do legado.

BC-07: Precificação (Tabelas de Preço)

> **Subdomínio:** Precificação e Modelo Comercial (Core Domain)
> **Package alvo:** `pk_precificacao`
> **Tabelas:** `tb_preco_plano`, `tb_valor_plano`, `tb_desconto_preco_plano`, `tb_parametro_franquia`
> **Responsabilidade:** Criar tabelas de preço, valores de planos e descontos.

Regras de Negócio

#	Código	Regra	Detalhamento
1	RN-07.01	**Criação de tabela de preço principal**	Quando `wfl_tabela_geral = 'N'`, o `lcd_tabela` é gerado por `fn_cd_tb_preco_plano` (function que encontra próximo código disponível → substitui o loop antigo de 1 a 999999). INSERT em `tb_preco_plano(cd_tabela, ds_tabela, fl_status, cd_um_corr, dt_validade, pc_desconto)` com: `ds_tabela = 'Tabela empresa ' || SUBSTR(lcd_empresa, 1, 5)`, `fl_status = 2` (ativo), `cd_um_corr = 1`. Se `WHEN OTHERS` ? log + `raise_application_error`. **Ref:** linhas ~1495-1535 do legado. **DS05:** `CalculoTabelaPrecoService`.
2	RN-07.02	**Criação de tabela de preço para agregados**	Só criada se `st_e.fl_agregados = 'S'` **E** existem registros em `tb_empresa_neg_tabela WHERE nu_controle = lnu_controle AND fl_tipo_tabela = 2`. O `lcd_tabela_agregado` é gerado por `fn_cd_tb_preco_plano`. INSERT idêntico ao principal, mas com `ds_tabela = 'Tabela de precos agregados ' || lcd_empresa`. **Ref:** linhas ~1710-1760 do legado.

#	Código	Regra	Detalhamento
3	RN-07.03	**Cópia de valores do modelo de negócio**	**FOR cursor:** `SELECT cd_plano, qt_empregados_inicial, qt_empregados_final, qt_idade_inicial, qt_idade_final, cd_taxa, vl_plano FROM tb_registro_plano_ans r, tb_plano_ans p, tb_empresa_neg_tabela et WHERE et.nu_controle = lnu_controle AND (et.fl_pim = 0 OR et.fl_pim IS NULL) AND et.fl_tipo_tabela = 1 AND p.cd_plano = et.cd_plano AND r.nu_reg_plano_ans = p.nu_reg_plano_ans AND UPPER(r.fl_participacao) = UPPER(NVL(st_e.fl_coparticipacao, 'N'))`. O INSERT em `tb_valor_plano` aplica o fator: `vl_plano = st_preco.vl_plano * constante_vl`, onde `constante_vl = 1 - (pc_desconto/100)`. **Filtro ANS:** só planos com `fl_participacao` compatível com a flag de coparticipação. **Ref:** linhas ~1545-1620 do legado.
4	RN-07.04	**Normalização de faixa etária pós-insert**	O `lfl_tipo_faixa` é obtido de `tb_empresa_neg.fl_tipo_faixa` durante o loop de `st_dados` (linhas ~1915-1920). Este valor controla a faixa etária ANS aplicada nos valores: `v_fl_faixa_ans := 'S'` quando `fn_checa_contrato_affix = 'S'` OU `wcd_canal_venda IN (1, 2)` OU `nu_total_empregado = 100`. A flag `fl_faixa_etaria_ans` é inserida na `tb_empresa_conveniada`.
5	RN-07.05	**Inserção de descontos por faixa de usuários**	**Condição:** `st_e.fl_descontos = 'S' AND wfl_tabela_geral = 'N'`. Para BITIX, `fl_descontos` é forçada para `'S'` (`IF V_COUNT_BITIX > 0 AND st_e.fl_descontos = 'N' THEN st_e.fl_descontos := 'S'`). **FOR cursor:** `SELECT qt_usuario_inicial, qt_usuario_final, pc_desconto, pc_desconto_dif FROM tb_empresa_neg_desconto WHERE nu_controle = lnu_controle AND fl_tipo_tabela = 1`. INSERT em `tb_desconto_preco_plano(cd_tabela, qt_usuario_inicial, qt_usuario_final, pc_desconto, pc_desconto_dif)`. **Ref:** linhas ~1665-1700 do legado.
6	RN-07.06	**Inserção de franquias por plano**	**FOR cursor:** `SELECT cd_plano, nu_mes_inicial, nu_mes_final, vl_franquia FROM tb_empresa_neg_franquia WHERE nu_controle = lnu_controle`. INSERT em `tb_parametro_franquia(cd_empresa_coneveniada, cd_plano, nu_mes_inicial, nu_mes_final, vl_franquia)`. ⚠ **Bug:** nome da coluna no legado ? `cd_empresa_coneveniada` (typo). `EXCEPTION WHEN OTHERS THEN NULL` ? falhas silenciadas. **Ref:** linhas ~1640-1665 do legado.
7	RN-07.07	**Filtro de status e coparticipação**	Na query de `tb_empresa_neg_tabela`, os registros são filtrados por: `(fl_pim = 0 OR fl_pim IS NULL)` ? exclui registros PIM; `UPPER(r.fl_participacao) = UPPER(NVL(st_e.fl_coparticipacao, 'N'))` ? seleciona planos compatíveis (com ou sem coparticipação conforme a proposta). Registros de `tb_valor_plano` são inseridos com `fl_status = 2` (ativo). O INSERT do valor só ocorre se `lcd_tipo_acomodacao IS NOT NULL`.
8	RN-07.08	**Reutilização de tabela geral (AFFIX)**	Quando `wfl_tabela_geral = 'S'` (modelo != 1 e `lcd_tabela_saude IS NOT NULL`), toda a criação de `tb_preco_plano`, `tb_valor_plano`, `tb_desconto_preco_plano` e `tb_parametro_franquia` é **ignorada** ? o fluxo simplesmente define `lcd_tabela := lcd_tabela_saude` (tabela já existente). Isso garante que contratos AFFIX da mesma filial compartilhem a mesma tabela de preço, permitindo reajuste único. **Sacti:** 355061 (David Brandão, 03/05/2016).

BC-08: Empresa Conveniada (Contrato)

> **Subdomínio:** Efetivação de Contrato PJ (Core Domain)
> **Package alvo:** `pk_empresa_conveniada`
> **Tabelas:** `tb_empresa_conveniada`, `tb_empresa_conveniada_unidade`, `tb_parametros_unidade`, `tb_parametros_unidade_pln_esp`, `tb_empresa_conveniada_nat`, `tb_emp_conveniada_saude_flags`, `tb_modelo_reaj_empresa`, `tb_implantacao_emp`, `tb_empresa_conveniada_com`, `tb_empresa_cnpj_contratante`, `tb_empresa_conveniada_imagem`, `tb_hist_empresa_conveniada`, `tb_acesso_dados_empresa`
> **Responsabilidade:** Criação do registro principal do contrato e todas as configurações associadas.

Regras de Negócio

#	Código	Regra	Detalhamento
1	RN-08.01	**Geração de código único de empresa**	O `lcd_empresa` é gerado por `fn_empresa_conveniada()` em loop: `WHILE (lcd_empresa IS NULL) AND (V_COUNT_EMP < 10001) LOOP lcd_empresa := fn_empresa_conveniada(); V_COUNT_EMP := V_COUNT_EMP + 1; END LOOP`. A function retorna `VARCHAR2(7)` ? código alfanumérico com `LPAD('0', 5)`. Se após 10.001 tentativas não encontrar código disponível, `lcd_empresa` permanece `NULL` e o fluxo falha downstream. ⚠ **Bug de design:** o código é gerado **ANTES** das validações (linha ~243), desperdiçando códigos quando a proposta é rejeitada. **Ref:** linhas ~241-245 do legado.

#	Código	Regra	Detalhamento
2	RN-08.02	**INSERT de empresa conveniada (~60 colunas)**	Existem **dois INSERTs quase idênticos** controlados por `IF st_e.dt_inicio <= sysdate AND V_COUNT_DT_ASSINATURA > 0` (BITIX com data no passado): o primeiro (linhas ~2010-2140) ajusta `dt_dia_pagamento`, `dt_processamento`, `dt_referencia_carencia` para `TRUNC(SYSDATE)`; o segundo (linhas ~2150-2300) usa `st_e.dt_inicio` original. Ambos inserem ~60 colunas incluindo: `cd_pessoa`, `cd_empresa_conveniada`, `nu_total_empregado`, `cd_vendedor`, `cd_plano`, `cd_tipo_acomodacao`, `cd_tabela`, `cd_filial`, `fl_natureza_empresa`, `fl_status = 2`, `cd_canal_venda`, etc. **Ref:** linhas ~2010-2300 do legado.
3	RN-08.03	**Status inicial ativo**	A empresa é criada com `fl_status = 2` (ativo). Valores possíveis: `1` = pré-pagamento, `2` = ativo, `3` = cancelado. O status inicial é hardcoded no INSERT.
4	RN-08.04	**Empresa deve ter ao menos uma unidade contratual**	INSERT em `tb_empresa_conveniada_unidade(cd_empresa_conveniada, cd_unidade)` com `cd_unidade = 1` (unidade padrão). Todo contrato PJ tem pelo menos a unidade 1. Unidades adicionais são criadas manualmente após a efetivação. **Ref:** linhas ~2450-2460 do legado.
5	RN-08.05	**Configuração de parâmetros de unidade**	FOR loop de `fl_tipo_tabela` de `0` a `8`, criando registros em `tb_parametros_unidade(cd_empresa_conveniada, cd_unidade, fl_tipo_tabela, cd_tabela)`. Para `fl_tipo_tabela = 1` (titular) ? `cd_tabela = lcd_tabela`; para `fl_tipo_tabela = 2` (agregado) ? `cd_tabela = NVL(lcd_tabela_agregado, lcd_tabela)`. Demais tipos recebem `NULL`. **Ref:** linhas ~2470-2520 do legado.
6	RN-08.06	**Parâmetros de plano especial**	**FOR cursor:** `SELECT * FROM tb_empresa_neg_pln_esp WHERE nu_controle = lnu_controle`. INSERT em `tb_parametros_unidade_pln_esp(cd_empresa_conveniada, cd_unidade, cd_plano_especial, ...)`. Aplicável quando o modelo de negócio define planos com condições especiais (ex: plano executivo com tabela diferenciada).
7	RN-08.07	**Natureza da empresa conveniada**	INSERT em `tb_empresa_conveniada_nat(cd_empresa_conveniada, fl_natureza_empresa, dt_inicio)`. Registra a natureza jurídica do contrato (PME=6, Simples=9, etc.) com data de início. Permite histórico de alterações de natureza.
8	RN-08.08	**Flags de saúde**	INSERT em `tb_emp_conveniada_saude_flags(cd_empresa_conveniada, ...)` com parâmetros regulamentados. Inclui flags de RN 195 ANS (`fl_modelo_rn195 = 2`), `dt_rn195 = TRUNC(st_e.dt_inicio)`, e flags de controle de faturamento/utilização.
9	RN-08.09	**Modelo de reajuste**	INSERT em `tb_modelo_reaj_empresa(cd_modelo_reaj_empresa, cd_indice_modelo_calculo, cd_empresa_conveniada, ...)` usando `v_id_indice_empresa` e `v_cd_indice_modelo_calculo` obtidos de `tb_empresa_neg_controle`. Define a regra de reajuste anual do contrato (Índice de correção).
10	RN-08.10	**Implantação de empresa**	INSERT em `tb_implantacao_emp(cd_empresa_conveniada, dt_implantacao, fl_status_implantacao, ...)`. Registra que a empresa foi implantada automaticamente pelo processo de "baixa". O campo `vl_implantacao` (`p_vl_implantacao`) é calculado a partir dos valores do plano.
11	RN-08.11	**Comissão de consultor**	**Condição:** verifica se existem registros em `tb_empresa_internet_com WHERE nu_controle = :controle`. Se sim, copia para `tb_empresa_conveniada_com(cd_empresa_conveniada, cd_consultor, pc_comissao, ...)`. **Ref:** linhas ~2600-2630 do legado.
12	RN-08.12	**CNPJ contratante (AFFIX)**	**Condição:** `IF fn_checa_contrato_affix(p_nu_controle, 'S') = 'S'` ? INSERT em `tb_empresa_cnpj_contratante(cd_empresa_conveniada, nu_cnpj, cd_pessoa_contratante, ...)` vinculando o CNPJ da empresa contratante ? empresa AFFIX. Permite que a administradora AFFIX gerencie contratos de terceiros.
13	RN-08.13	**Cálculo de canal de venda**	Cálculo em 3 etapas com override: (1) individual: `1-29 -> 1, 30-99 -> 2, >=100 -> NULL`; (2) BITIX coligada: recalcula com `V_NU_TOTAL_BITIX`; (3) SIGO coligada: recalcula com `NU_TOTAL_EMPREGADO_SIGO`. O último cálculo prevalece. **Value Object:** `CanalVenda` (VO07). **Ref:** linhas ~1875-1910 do legado.
14	RN-08.14	**Variante BITIX: data de início**	**Condição:** `IF st_e.dt_inicio <= sysdate AND V_COUNT_DT_ASSINATURA > 0` (→ BITIX). Neste caso: `dt_dia_pagamento := fn_buscar_dia_pagamento(TRUNC(SYSDATE))` em vez de `st_e.dt_inicio`; `dt_processamento := TRUNC(SYSDATE)`; `dt_referencia_carencia := TRUNC(SYSDATE)`. A `dt_validade_contrato` ? ajustada: se <= `ADD_MONTHS(SYSDATE, 12)` ? `ADD_MONTHS(TRUNC(SYSDATE), 12)`. **Ref:** linhas ~1990-2010 do legado.

#	Código	Regra	Detalhamento
15	RN-08.15	**Histórico somente para PME/Simples**	**Condição:** `IF st_e.fl_natureza_empresa IN ('6', '9')` ? INSERT em `tb_hist_empresa_conveniada(cd_empresa_conveniada, cd_assunto, dt_historico, ds_observacao)` com `cd_assunto = 130` ("Empresa Nova"). Naturezas 6 (PME) e 9 (Simples) recebem este histórico para rastreamento regulatório. Demais naturezas não geram registro.
16	RN-08.16	**Configuração RN 279/309 (tabela inativos)**	Chama `pr_tabela_inativos_rn279(lcd_empresa, dt_inicio, lcd_filial, fl_natureza_empresa, nu_total_empregado, cd_modelo_negocio, ...)` que retorna `lcd_tabela_inativo`, `lnu_controle_inativos`, `lfl_atende_rn279`. Se tabela encontrada, copia via `fn_copia_tabela_preco(lcd_tabela_inativo)` ? `lcd_tabela_copia_inativo`. Se cópia OK e atende RN279: `ldt_rn_279_309_2 := TRUNC(st_e.dt_inicio)`, `lfl_rn_279_309 := 2`. Inseridos na `tb_empresa_conveniada`. Conformidade ANS. **Ref:** linhas ~1950-1990 do legado.
17	RN-08.17	**Imagem do contrato**	UPDATE em `tb_empresa_conveniada_imagem SET ... WHERE cd_empresa_conveniada = lcd_empresa`. O registro é atualizado (não inserido) com a imagem/documento do contrato quando disponível.
18	RN-08.18	**Dia de pagamento convertido**	`fn_buscar_dia_pagamento(dt)` converte o dia bruto (1-5) para dia real: `1->5, 2->10, 3->15, 4→20, 5→25`. **Duas variantes:** BITIX com `dt_inicio <= sysdate` usa `fn_buscar_dia_pagamento(TRUNC(SYSDATE))`; demais usam `fn_buscar_dia_pagamento(st_e.dt_inicio)`. O resultado é inserido em `tb_empresa_conveniada.dt_dia_pagamento`. **Ref:** linhas ~2030 e ~2180 do legado.

BC-09: Coparticipação e Fator Empresa

> **Subdomínio:** Regulamentação e Participação do Beneficiário (Core Domain)
> **Package alvo:** `pk_coparticipacao`
> **Tabelas:** `tb_controle_fator_empresa`, `tb_fator_empresa`, `tb_terapias_espec_empresa`, `tb_copart_internacao_param_pj`, `tb_copart_isnt_tp_benef`, `tb_copart_tipo_reembolso`, `tb_copart_tab_empresa`
> **Responsabilidade:** Configurar regras de coparticipação, franquias, fatores e isenções. Regulamentado pela ANS.

Regras de Negócio

#	Código	Regra	Detalhamento
1	RN-09.01	**Criação de controle fator empresa**	INSERT em `tb_controle_fator_empresa` com ~23 colunas: `cd_empresa_conveniada`, `dt_inicio`, `fl_natureza_fator`, `fl_cobra_rede_propria`, `fl_cobra_retorno`, `fl_cobra_urgencia`, `fl_cobra_pericultura`, `fl_cobra_gineco`, `fl_valida_custo`, `fl_cobra_internacao_urgencia`, `fl_cobra_internacao_exame = 'N'`, `fl_cobra_odonto`, `fl_cobra_med_prev`, `fl_cobra_seg_opiniao`, `fl_cobrar_consulta_exame = 'S'`, `vl_limite`, `vl_exame_complexo`, `dt_ref_reajuste_fator = TRUNC(ADD_MONTHS(V_DT_PROCESSAMENTO, 11), 'mm')`, `FL_COBRA_PRODUTO = DECODE(fl_natureza_empresa, '6', 'N', '9', 'N', 'S')` ? PME/Simples não cobra produto. Dados originam de `tb_empresa_neg_controle WHERE nu_controle = lnu_controle`. **Ref:** linhas ~2830-2920 do legado.
2	RN-09.02	**Duas variantes: AFFIX vs Normal**	Controlado por `IF fn_checa_contrato_affix(p_nu_controle, 'S') = 'S'`. **AFFIX:** INSERT inclui `cd_empresa_cobranca_copart = st_e.cd_empresa_cobranca` e `fl_referencia_reajuste = 'ATENDIMENTO'`. **Normal:** INSERT sem `cd_empresa_cobranca_copart`. Ambos usam `fl_fator_padrao = NVL(reg.fl_fator_padrao, DECODE(wcd_empresa_plano, 14, 'NDI SP', NULL))` ? NDI SP tem fator padrão específico. **Sacti:** 1572517. **Ref:** linhas ~2820 (AFFIX) e ~2870 (Normal) do legado.
3	RN-09.03	**Fatores por faixa etária e franquia**	**FOR cursor:** `SELECT cd_fator, qt_inicial, qt_final, qt_franquia_inicial, qt_franquia_final, pc_participacao, vl_parcipacao, fl_utilizacao, cd_fator_referencia, fl_periodo, pc_administrativo, qt_idade_inicial, qt_idade_final, qt_tempo_inicial, qt_tempo_final, nu_perc_revertido, fl_urgencia_eletivo, qt_pericultura_isento, qt_gravida_isento, vl_limite, NVL(cd_rede, 'T'), NVL(vcd_tabela_fator_moderador, DECODE(wcd_empresa_plano, 14, 4, 1)), cd_tipo_rede_atendimento, fl_complexo, cd_plano FROM tb_empresa_neg_fator WHERE nu_controle = lnu_controle`. INSERT em `tb_fator_empresa` com 27 colunas. `dt_opcao_fm = V_DT_PROCESSAMENTO`. `EXCEPTION WHEN OTHERS THEN NULL`. **Ref:** linhas ~2930-3050 do legado.
4	RN-09.04	**Terapias especiais**	**Condição:** `IF NVL(reg.fl_isenta_terapias_especiais, 'N') = 'S'`. FOR cursor em `tb_terapias_espec_emp_padrao` (tabela de templates padrão). Para cada terapia, verifica se já existe: `SELECT 'X' FROM tb_terapias_espec_empresa WHERE cd_empresa_conveniada = lcd_empresa AND cd_terapia = :terapia`. Se `NO_DATA_FOUND` ? INSERT. Evita duplicatas. **Sacti:** 1288539 (Marcelo Bissoni Jr). **Ref:** linhas ~2940-2960 do legado.

#	Código	Regra	Detalhamento
5	RN-09.05	**Isenção de coparticipação**	**FOR cursor:** `SELECT DISTINCT tp_franquia, nu_idade_ini_isenta, nu_idade_fim_isenta, fl_reembolso, fl_isenta_cronico, fl_revertido, fl_isenta_cc, fl_isenta_homecare, fl_isenta_demitido, fl_tipo_benef_cobranca, fl_isenta_aposentado FROM tb_empresa_neg_isenta_copart, tb_empresa_neg_tabela WHERE nu_controle = lnu_controle`. **UPDATE** em `tb_controle_fator_empresa SET tp_franquia = :val, nu_idade_ini_isenta = :val, ...` (11 campos atualizados). `EXCEPTION WHEN OTHERS THEN NULL`. **Ref:** linhas ~4080-4110 (pós-carência, seção "Isenção de carência Daniel Evangelista 09/2024").
6	RN-09.06	**Internação PJ**	**FOR cursor:** `SELECT * FROM tb_empresa_cop_intern_pj WHERE nu_controle = lnu_controle`. INSERT em `tb_copart_internacao_param_pj(cd_empresa_conveniada, cd_plano, fl_tipo_teto_internacao, vl_fixo_internacao, vl_teto_internacao, pc_internacao, fl_rede, dt_cadastro = SYSDATE, cd_operador = USER, dt_inicio, dt_fim)`. Se pelo menos 1 registro inserido, `vfl_intern := 'S'`. **Ref:** linhas ~4130-4170 do legado.
7	RN-09.07	**Flag de cobrança de internação**	**Condição:** `IF vfl_intern = 'S'` → `UPDATE tb_controle_fator_empresa SET fl_cobra_internacao = 'S' WHERE cd_empresa_conveniada = lcd_empresa`. A flag é derivada ? só ativada quando existem parâmetros de internação PJ. `EXCEPTION WHEN OTHERS THEN NULL`. **Ref:** linhas ~4175-4185 do legado.
8	RN-09.08	**Isenção por tipo de beneficiário**	**FOR cursor:** `SELECT * FROM tb_empresa_neg_isnt_cop_tp_ben WHERE nu_controle = lnu_controle`. INSERT em `tb_copart_isnt_tp_benef(nu_registro = (SELECT NVL(MAX(nu_registro)+1, 1) FROM TB_CONFIG_ISNT_COP_TP_BENEF), cd_empresa_conveniada, cd_tp_dependente, fl_ativo, cd_operador = USER, dt_cadastro = SYSDATE)`. ⚠ **Problema:** `MAX(nu_registro)+1` gera race condition (sem sequence). **Ref:** linhas ~4190-4215 do legado.
9	RN-09.09	**Tipo de reembolso coparticipação**	**FOR cursor:** complex join de `tb_empresa_neg_isenta_copart` com `tb_valor_plano`, `tb_plano`, `tb_plano_ans`, `tb_registro_plano_ans`, `tb_reg_ans_parametro` filtrando por `fl_participacao = 'S'`, `fl_livre_escolha = 'S'`, `fl_status NOT IN (1,3)`, `cd_tabela = lcd_tabela`. INSERT em `tb_copart_tipo_reembolso(cd_empresa_conveniada, cd_plano, cd_tipo_reembolso, dt_cadastro, cd_operador, fl_ativo = 'S')`. `DUP_VAL_ON_INDEX` ? UPDATE. **Ref:** linhas ~4220-4260 do legado.
10	RN-09.10	**Tabela de referência coparticipação**	**FOR cursor:** `SELECT DISTINCT cd_tabela, cd_plano, dt_inicio, dt_fim FROM tb_copart_tabela_plano t, tb_valor_plano v, ... WHERE t.cd_plano = v.cd_plano AND st_e.dt_inicio BETWEEN t.dt_inicio AND NVL(t.dt_fim, SYSDATE+3000) AND v.cd_tabela = lcd_tabela`. INSERT em `tb_copart_tab_empresa(nu_registro = (SELECT NVL(MAX(nu_registro)+1, 1)), cd_empresa_conveniada, cd_tabela, cd_plano, dt_inicio, dt_fim, cd_operador = USER, dt_cadastro = SYSDATE)`. ⚠ **Problema:** `MAX+1` sem sequence. **Ref:** linhas ~4270-4310 do legado.
11	RN-09.11	**Isenção de carência (Daniel Evangelista 09/2024)**	Seção adicionada em setembro/2024 (comentário: "Isenção de carência — Daniel Evangelista 09/2024"). Agrupa as regras RN-09.05, RN-09.06, RN-09.07, RN-09.08, RN-09.09, RN-09.10 em um bloco lógico coeso. Inclui isenção por tipo de beneficiário, tipo de reembolso e tabela de referência coparticipação. Finalizado com comentário "Fim isencao carencia Daniel Evangelista." **Ref:** linhas ~4080-4320 do legado.
12	RN-09.12	**Coparticipação regulamentada pela ANS**	BC-09 → **Core Domain** por regulamentação ANS (Agência Nacional de Saúde Suplementar). Erros na coparticipação geram: glosas em procedimentos, multas regulatérias da ANS, processos judiciais de beneficiários, impacto financeiro direto. O BC-09 possui ~1.300 linhas no legado (linhas 2800-4100), sendo o **maior bloco** da procedure. Exige testes extensivos na refatoração com cobertura de ~95%.

BC-10: Carência e Compra de Carência

> **Subdomínio:** Regulamentação e Participação do Beneficiário (Core Domain)
> **Package alvo:** `pk_carencia`
> **Tabelas:** `tb_compra_carencia`, `tb_compra_grupo`, `tb_compra_modulo`, `tb_odon_compra_grupo`
> **Responsabilidade:** Configurar regras de carência, compra de carência, grupos e módulos.

Regras de Negócio

#	Código	Regra	Detalhamento

#	Código	Regra	Detalhamento
1	RN-10.01	**Criação de compra de carência**	`SELECT sq_compra_carencia.NEXTVAL INTO lnu_ordem_compra FROM dual`. INSERT em `tb_compra_carencia(nu_ordem_compra, cd_empresa_conveniada, dt_compra, tp_compra, cd_operador, dt_cadastro, fl_ativo)` com `cd_empresa_conveniada = lcd_empresa`, `dt_compra = st_e.dt_inicio`, `cd_operador = USER`, `fl_ativo = 'S'`. Cada compra de carência é um lote que agrupa vários grupos e módulos de carência. **Ref:** linhas ~3730-3760 do legado.
2	RN-10.02	**Dados oriundos do staging de negociação**	**FOR cursor:** `SELECT * FROM tb_empresa_neg_carencia WHERE nu_controle = lnu_controle`. O `lnu_controle` ? o modelo de negócio resolvido (BC-06). Cada registro de staging gera uma compra de carência com seus filhos. Os campos copiados incluem: `tp_compra`, datas, e percentuais de compra.
3	RN-10.03	**Criação de grupos de carência**	**FOR cursor:** `SELECT cd_carencia, pc_compra FROM tb_empresa_neg_grupo WHERE nu_ordem_compra = st_compra.nu_ordem_compra`. INSERT em `tb_compra_grupo(nu_ordem_compra, cd_carencia, dias_cumprir = NULL, pc_compra)`. O `nu_ordem_compra` ? o novo (gerado por sequence), não o do staging. `EXCEPTION WHEN OTHERS` ? log + raise. **Ref:** linhas ~3780-3830 do legado.
4	RN-10.04	**Criação de módulos de carência**	**FOR cursor:** `SELECT cd_modulo, pc_compra FROM tb_empresa_neg_modulo WHERE nu_ordem_compra = st_compra.nu_ordem_compra`. INSERT em `tb_compra_modulo(nu_ordem_compra, cd_modulo, dias_cumprir = NULL, pc_compra, qt_usuarios_inicial = 0, qt_usuarios_final = 999)`. A faixa de usuários `0-999` abrange todos os cenários. `EXCEPTION WHEN OTHERS` ? log + raise. **Ref:** linhas ~3840-3890 do legado.
5	RN-10.05	**Criação de grupos odontológicos**	**FOR cursor:** `SELECT cd_grupo, dias_cumprir FROM tb_odon_empresa_neg_grupo WHERE nu_ordem_compra = st_compra.nu_ordem_compra`. INSERT em `tb_odon_compra_grupo(nu_ordem_compra, cd_grupo, dias_cumprir, cd_operador = USER, dt_operacao = SYSDATE)`. Diferente dos outros, este INSERT preserva `dias_cumprir` (dias para cumprir carência odonto). **Sacti:** 981015. **Ref:** linhas ~3900-3950 do legado.
6	RN-10.06	**Grupos e módulos são filhos da compra**	O `lnu_ordem_compra` (novo, gerado por sequence) é usado como FK em todos os INSERTs filhos: `tb_compra_grupo`, `tb_compra_modulo`, `tb_odon_compra_grupo`. Formam a hierarquia: `tb_compra_carencia` ? `tb_compra_grupo` + `tb_compra_modulo` + `tb_odon_compra_grupo`. A exclusão em cascata exige DELETE em ordem inversa.
7	RN-10.07	**Carência regulamentada pela ANS**	Carência é regulada pela Lei 9.656/98 e RN 162/2007. A ANS define carências máximas: 24h (urgência/emergência), 180 dias (internação), 300 dias (parto), 720 dias (doenças pré-existentes). A "compra de carência" permite redução ou eliminação mediante pagamento adicional (percentual `pc_compra`). O módulo define o tipo de carência (ambulatorial, hospitalar, etc.) e o grupo define procedimentos específicos.

BC-11: Fidelização Contratual

> **Subdomínio:** Termos Contratuais (Supporting)
> **Package alvo:** `pk_fidelizacao`
> **Tabelas:** `tb_fidelizacao_empresa`
> **Responsabilidade:** Configurar regras de fidelização para contratos do segmento Middle Market.

Regras de Negócio

#	Código	Regra	Detalhamento
1	RN-11.01	**Fidelização somente para canal Middle (30-99 vidas)**	**Condição:** `IF wcd_canal_venda = 2` (Middle Market). O bloco inteiro de fidelização é envolvido neste IF. Canais 1 (varejo PIM) e NULL (grandes contas) não geram registros em `tb_fidelizacao_empresa`. Justificativa: contratos Middle possuem condições comerciais diferenciadas que justificam exigência de permanência mínima. **Ref:** linhas ~3170 do legado.
2	RN-11.02	**Exclusão de odontologia**	**Filtro no cursor:** `AND fl_atendimento != 4` ? registros com atendimento odontológico puro (tipo 4) são excluídos. A fidelização se aplica apenas a planos de saúde e mistos.

#	Código	Regra	Detalhamento
3	RN-11.03	**Data fim = data início + 1000 dias**	INSERT em `tb_fidelizacao_empresa` com `dt_fim = st_e.dt_inicio + 1000`. 1000 dias ≈ 2 anos e 9 meses. O valor é hardcoded no INSERT, sem parametrização. A `dt_inicio` é a mesma da proposta (`st_e.dt_inicio`). Saída antecipada (antes de `dt_fim`) pode gerar multa contratual. **Ref:** linhas ~3185-3195 do legado.
4	RN-11.04	**Dados do modelo de negócio**	**FOR cursor:** `SELECT cd_tipo_fidelizacao, fl_atendimento, nu_dias_fidelizado, nu_admissao_inicial, nu_admissao_final FROM tb_empresa_neg_fidelizacao WHERE nu_controle = lnu_controle AND fl_atendimento != 4`. Cada registro gera um INSERT em `tb_fidelizacao_empresa` com os mesmos campos + `cd_empresa_conveniada = lcd_empresa`. `EXCEPTION WHEN OTHERS THEN NULL`.
5	RN-11.05	**Período mínimo de permanência**	A fidelização define: `cd_tipo_fidelizacao` (tipo de regra), `nu_dias_fidelizado` (dias de fidelização específicos), `nu_admissao_inicial` e `nu_admissao_final` (faixa de admissão elegível). Beneficiários admitidos entre essas datas estão sujeitos à regra. O campo `nu_dias_fidelizado` pode diferir dos 1000 dias do `dt_fim` ? é usado para cálculos de multa proporcional.

BC-12: Acesso Internet / Portal

> **Subdomínio:** Acesso e Identidade (Generic)
> **Package alvo:** `pk_acesso_internet`
> **Tabelas:** `tb_acesso_internet`, `tb_controle_internet`, `tb_emp_limite_acesso_contra`, `tb_numero_extenso`
> **Responsabilidade:** Criar acessos internet, controles de movimentação e credenciais.

Regras de Negócio

#	Código	Regra	Detalhamento
1	RN-12.01	**Geração de código de acesso**	`select sq_acesso_internet.nextval into lcd_acesso from dual` (~linha 4376). O `cd_acesso` é gerado via sequence dedicada e torna-se a PK do registro de acesso. Se INSERT falhar com `DUP_VAL_ON_INDEX`, lcd_acesso ? zerado (null) e o fluxo tenta recuperar o acesso existente via CNPJ+empresa_plano.
2	RN-12.02	**Geração de senha**	Senha = `pk_administracao.fn_encripta(substr(to_char(lcd_pessoa), (length(lcd_pessoa) - 5), 6), 1, 10)` (~linha 4383). Extrai os **últimos 6 dígitos** de `lcd_pessoa`, encripta com algoritmo proprietário (modo 1, tamanho 10). A mesma senha ? incluída no corpo do e-mail de efetivação (RN-17.01) como `substr(to_char(lcd_pessoa),(length(lcd_pessoa) - 5),6)` ? ou seja, o e-mail envia a **senha em texto claro** antes da encriptação. ⚠ **Risco de segurança a tratar na refatoração.**
3	RN-12.03	**Tipo de acesso fixo**	INSERT em `tb_acesso_internet` com `cd_tipo_acesso = 5` (empresa conveniada), `fl_status = 1` (ativo), `fl_email_atualizado = 1`. Valores hardcoded (~linha 4380). Na refatoração, externalizar como constantes/enum `TipoAcesso.EmpresaConveniada`.
4	RN-12.04	**Criação de controles para 4 serviços**	Loop `for st_controle in (select qt_numero from tb_numero_extenso where qt_numero in (7, 12, 14, 16))` (~linha 4411). INSERT em `tb_controle_internet` com `cd_acesso`, `cd_servico`, `fl_status = 1`, `dia_limite_acesso`. Serviço 7 = movimentação de beneficiários; 12, 14, 16 = outros serviços do portal. Se `lcd_acesso` for NULL (acesso duplicado), tenta recuperar via `select cd_acesso from tb_acesso_internet where cd_pessoa = (subquery CNPJ+empresa_plano) and cd_tipo_acesso = 5` e faz UPDATE no `dia_limite_acesso` ao invés de INSERT.
5	RN-12.05	**Dia limite baseado no dia de pagamento**	Cálculo condicional (~linha 4416): `IF st_e.dt_dia_pagamento = 5 AND cd_servico = 7 THEN wdia_limite := 10; ELSIF dt_dia_pagamento <> 5 AND cd_servico = 7 THEN wdia_limite := 15; ELSE wdia_limite := 31`. O dia limite só é diferenciado para o serviço **7** (movimentação). Para serviços 12, 14 e 16, sempre `dia_limite_acesso = 31`. Valor fallback: `nvl(wdia_limite, 31)`.
6	RN-12.06	**Tratamento AFFIX: cópia de limites**	Guarda `fn_checa_contrato_affix(p_nu_controle, 'S') = 'S'` (~linha 4474). INSERT-SELECT de `tb_emp_limite_acesso_contra` da empresa de cobrança (`st_e.cd_empresa_cobranca`) para a nova empresa. Colunas copiadas: `cd_acesso`, `cd_servico`, `fl_status` (apenas `fl_status = 1`), `dia_limite_acesso`. SACTI 868454 ? [CAFEX]-MOVIMENTAÇÃO - DATA CORTE POR CONTRATO (Thompson Dantas).
7	RN-12.07	**Atualização de titulares/dependentes internet**	Dois caminhos (~linha 4498): **Se BITIX** (`dt_inicio <= sysdate AND V_COUNT_DT_ASSINATURA > 0`): UPDATE com `cd_empresa = lcd_empresa`, `cd_vendedor_plano`, `dt_inicio = V_DT_PROCESSAMENTO` (ajusta data). **Senão**: UPDATE apenas `cd_empresa` e `cd_vendedor_plano`. Ambos mudam de `'T' || P_NU_CONTROLE` para `lcd_empresa` (provisório ? definitivo). Aplica-se a `tb_usuario_titular_internet` e `tb_usuario_dependente_internet` separadamente.

#	Código	Regra	Detalhamento
8	RN-12.08	**Número extenso para controle**	`tb_numero_extenso` usada como tabela auxiliar de enumeração: `where qt_numero in (7, 12, 14, 16)` para iterar serviços e `where qt_numero in (0,1,2,3,4,5,6,7,8)` para tipos de usuário em BC-08. Padrão legado Oracle de usar tabela de números ao invés de coleção PL/SQL. Na refatoração, substituir por arrays/enums.
9	RN-12.09	**Acesso para portal web da empresa**	Acesso permite movimentação online (serviço 7), além de serviços 12, 14 e 16 do portal. Também é criado registro em `tb_acesso_dados_empresa` com `ds_email_adm = st_e.ds_endereco_eletronico` (~linha 4610) para vincular e-mail ao acesso. SACTI 1941776 (Roberto Santos).

BC-13: Integração Odontológica

> **Subdomínio:** Integração com Odontologia (Supporting)
> **Package alvo:** `pk_integracao_odonto`
> **Tabelas/Procedures:** `tb_odon_param_esp_operadora`, `tb_odon_parametro_diversos`, `pr_odon_param_esp_empresa`, `Pr_Vcc_Empresa_Super_Simples`
> **Responsabilidade:** Espelhar empresa para odontologia e configurar super simples. Protegido por ACL.

Regras de Negócio

#	Código	Regra	Detalhamento
1	RN-13.01	**Espelhamento condicionado ao parâmetro 225**	`select p.qt_referencia into vCd_ParametroEspelhamento from tb_odon_parametro_diversos p where p.cd_parametro_diversos = 225` (~linha 4862). Se `vCd_ParametroEspelhamento = 1` ? espelhamento ligado. Se `NO_DATA_FOUND` ? NULL ? espelhamento **desligado**. Parâmetro descrito como "Automatização de parametrização de espelhamento: Ligar/Desligar processo".
2	RN-13.02	**Espelhamento via procedure dedicada**	Chamada `pr_odon_param_esp_empresa(lcd_empresa)` (~linha 4888) que replica parâmetros do contrato saúde → odontologia. Só executa se `v_empresa_ope_automatica = 'S'`, ou seja, se existir parametrização automática para a operadora da empresa.
3	RN-13.03	**Super simples para empresas sem odonto**	`Humaster.Pr_Vcc_Empresa_Super_Simples(lcd_empresa)` (~linha 4910) ? chamado quando `wemp_odonto = 0`, ou seja, quando `count(1) = 0` na junção `tb_odon_empresa_internet oei, tb_empresa_internet ei WHERE oei.nu_controle_saude = ei.nu_controle AND ei.nu_controle = p_nu_controle AND fl_status_processamento NOT IN (2)`. Cria configuração mínima de odonto urgente para empresas que **não** possuem plano odonto. SACTI 1124736 - US 898.
4	RN-13.04	**Verificação de operadora automática**	Query complexa com 5 joins (~linha 4872): `tb_empresa_conveniada e, tb_filial f ? tb_odon_param_esp_operadora o ? tb_odon_param_esp_repasse t ? tb_odon_param_esp_produto p`. Verifica se existe parametrização ativa (`o.fl_ativo = 'S'`) para o `cd_empresa_plano` da filial. Se existir, seta `v_empresa_ope_automatica = 'S'`. LECOM 286080 - SACTI 1709171.
5	RN-13.05	**Execução pós-COMMIT**	Ambas as integrações (espelhamento e super simples) são executadas **após** o `COMMIT` da linha ~4856. Bloco DECLARE independente (~linha 4896) com `EXCEPTION WHEN OTHERS THEN NULL` ? falha na integração odonto **não** causa rollback. O contrato de saúde já está persistido.
6	RN-13.06	**Anti-Corruption Layer (ACL)**	O domínio saúde acessa tabelas `tb_odon_*` (prefixo odonto) apenas neste ponto (~linhas 4862-4920). As procedures `pr_odon_param_esp_empresa` e `Pr_Vcc_Empresa_Super_Simples` encapsulam toda lógica odontológica. Na refatoração, será um `IOdontologiaService` com chamada assíncrona via Domain Event `ContratoEfetivadoEvent` ? handler odonto.

BC-14: Auditoria e Log

> **Subdomínio:** Auditoria e Observabilidade (Generic)
> **Package alvo:** `pk_log_auditoria`
> **Tabelas:** `tb_log_baixa_controle`, `tb_pendencia_empresa_internet`
> **Responsabilidade:** Registrar logs de processamento, erros e pendências.

Regras de Negócio

#	Código	Regra	Detalhamento
1	RN-14.01	**Log de baixa controle**	Padrão repetido 30+ vezes: `select max(nvl(cb.cd_log, 0)) + 1 into wcd_log from humaster.tb_log_baixa_controle cb; INSERT INTO tb_log_baixa_controle (cd_log, nu_controle, ds_observacao, fl_status) VALUES (wcd_log, st_e.nu_controle, '<mensagem>', '15')`. Chave gerada por `MAX+1` (⚠ **race condition** em ambiente concorrente — na refatoração usar sequence ou GUID). `fl_status = '15'` indica erro de processamento. `fl_status = '9'` indica sucesso (~linha 4847). `ds_observacao` recebe `substr(sqlerrm, 1, 1024)` ou mensagem descritiva.
2	RN-14.02	**Log condicional a controle odonto**	Guard clause: `if nvl(wnu_controle_odonto, 0) > 0 then` antes de cada INSERT de log. Isso significa que contratos **sem** espelhamento odontológico **não geram log de erro** ? apenas um `raise_application_error` direto. Na refatoração, todo erro deve gerar log independente de odonto.
3	RN-14.03	**Registro de pendências**	No exception handler global (~linha 4942): `INSERT INTO TB_PENDENCIA_EMPRESA_INTERNET (NU_CONTROLE_PENDENCIA, CD_PENDENCIA, NU_CONTROLE, DT_STATUS, DS_OBSERVACAO, CD_OPERADOR) VALUES (SQ_CONTROLE_PENDENCIA.NEXTVAL, 9, p_nu_controle, SYSDATE, p_erro_controle, 'HUMASTER')`. `cd_pendencia = 9` = erro de processamento. Seguido de `COMMIT` explícito (persiste pendência mesmo após ROLLBACK do contrato). Permite reprocessamento automático pelo job.
4	RN-14.04	**Padrão de log repetido (problema)**	O bloco de log (~12 linhas) é copiado em **30+ locais** da procedure (~360 linhas de boilerplate). Mensagens variam: "informe DBA 1", "informe DBA 2", "informe DBA 3", "erro ao tentar incluir acesso internet", "erro ao tentar incluir controle internet", etc. Na refatoração: centralizar em `pk_log_auditoria.pr_registra_erro(p_nu_controle, p_nu_controle_odonto, p_contexto, p_mensagem)` → uma única chamada substitui 12 linhas.
5	RN-14.05	**Log não falha silenciosamente**	Cada bloco de log possui duplo `EXCEPTION WHEN OTHERS THEN NULL` → o primeiro captura erro do INSERT principal, e dentro deste bloco, o INSERT de log **também** tem `WHEN OTHERS THEN NULL`. O exception handler global (~linha 4926) faz `ROLLBACK` seguido de INSERT na `tb_pendencia_empresa_internet` e `COMMIT` do log. Além disso, verifica `count(1) from tb_empresa_conveniada WHERE cd_empresa_conveniada = lcd_empresa` ? se empresa já foi criada parcialmente, seta `p_erro_controle` com o código da empresa.

BC-15: Reembolso / Livre Escolha

> **Subdomínio:** Termos Contratuais (Supporting)
> **Package alvo:** `pk_reembolso`
> **Tabelas:** `tb_reemb_empresa_prazo_pag`, `tb_reemb_empresa_prazo_pag_tip`, `tb_reemb_empresa_tabela`, `tb_reemb_empresa_composicao`
> **Responsabilidade:** Configurar parâmetros de reembolso por plano.

Regras de Negócio

#	Código	Regra	Detalhamento
1	RN-15.01	**Somente para planos com livre escolha**	Guard clause: `if fn_plano_livre_escolha(cd_plano) = 'S' then` (~linha 4158). Loop `for rb in (select cd_plano from tb_empresa_neg where nu_controle = lnu_controle)` ? verifica **cada plano** do contrato. Se plano não possui livre escolha, pula para o próximo. Na refatoração, usar Specification `PlanoComLivreEscolhaSpec`.
2	RN-15.02	**Operação idempotente (DELETE + INSERT)**	Bloco DECLARE (~linha 4155) executa 4 DELETEs seguidos de INSERTs: `DELETE FROM tb_reemb_empresa_prazo_pag WHERE cd_empresa_conveniada = lcd_empresa AND cd_plano = rb.cd_plano`, idem para `_prazo_pag_tip`, `_tabela`, `_composicao`. Isso garante que reprocessamentos não dupliquem dados. Pattern "replace" com DELETE+INSERT ao invés de MERGE/UPSERT.
3	RN-15.03	**Cópia de 4 sub-tabelas**	4 loops cursor com INSERT-SELECT: (1) `prazo_pag(rb.cd_plano)` -> `tb_reemb_empresa_prazo_pag` (~linha 4195); (2) `tipo_prazo(rb.cd_plano)` -> `tb_reemb_empresa_prazo_pag_tip` (~linha 4221); (3) `tabela(rb.cd_plano)` -> `tb_reemb_empresa_tabela` (~linha 4253); (4) `composicao(rb.cd_plano)` -> `tb_reemb_empresa_composicao` (~linha 4296). Cada INSERT gera PK via `max(cd_reemb_empresa_*) + 1` (⚠ **race condition** — mesma issue do BC-14).

#	Código	Regra	Detalhamento
4	RN-15.04	**Dados originais do plano**	Cursores definidos como: `cursor prazo_pag(pcd_plano number) is select * from tb_reemb_plano_prazo_pag where cd_plano = pcd_plano`. Idem para `tipo_prazo`, `tabela`, `composicao` → fontes: `tb_reemb_plano_prazo_pag`, `tb_reemb_plano_prazo_pag_tip`, `tb_reemb_plano_tabela`, `tb_reemb_plano_composicao`. Colunas preservadas: `cd_formula_calculo`, `cd_reembolso`, `vl_moeda_cotacao`, `cd_tab_tax_serv`, `cd_tab_mat_med`, `vl_moeda_taxa`, `cd_tab_med`, `cd_tp_composicao`, `vl_cotacao_moeda_hon`, `vl_fator_tabela`, `vl_limite`, `vl_fixo`, `cd_moeda`, `vl_perc_recibo`, `dt_inicio`, `dt_fim`.
5	RN-15.05	**Reembolso vinculado à empresa e plano**	Cada INSERT recebe `lcd_empresa` + `d.cd_plano` como chave composta. O campo `cd_reemb_plano_*` (PK da tabela plano) é copiado como FK para rastreabilidade (`cd_reemb_plano_prazo_pag`, `cd_reemb_plano_tabela`, etc.). Operador = `USER`, data = `SYSDATE`.

BC-16: Mínimo Contratual e Breakeven

> **Subdomínio:** Termos Contratuais (Supporting)
> **Package alvo:** `pk_minimo_contratual`
> **Tabelas:** `tb_empresa_minimo_contratual`, `tb_param_minimo_contratual`, `tb_empresa_breakeven`
> **Responsabilidade:** Configurar mínimo contratual e breakeven.

Regras de Negócio

#	Código	Regra	Detalhamento
1	RN-16.01	**Busca de parâmetros vigentes**	Query (~linha 4823): `SELECT cd_minimo_contratual, dt_vigencia_ini, dt_vigencia_fim, vl_multa, vl_faturamento_minimo, qtd_vida, fl_tipo_minimo, fl_cobra_comissao, cd_plano, CASE WHEN cd_plano IS NULL THEN null ELSE lcd_tabela END AS cd_tabela FROM TB_PARAM_MINIMO_CONTRATUAL WHERE nu_controle = lnu_controle AND cd_ativo = 'S' AND DT_VIGENCIA_INI <= st_e.dt_inicio AND nvl(DT_VIGENCIA_FIM, st_e.dt_inicio) >= st_e.dt_inicio`. Três filtros: ativo + vigência início ? data contrato + vigência fim ? data contrato (com NVL para vigência aberta).
2	RN-16.02	**Mínimo contratual → opcional**	Exception handler: `WHEN NO_DATA_FOUND THEN NULL; WHEN OTHERS THEN NULL` (~linha 4854). Se não existir parametrização vigente, o contrato é criado normalmente sem mínimo contratual. Não gera erro nem pendência.
3	RN-16.03	**Geração de código via sequence**	INSERT usa `sq_empresa_minimo_contratual.nextval` (~linha 4842). Colunas: `cd_minimo_contratual` (PK sequence), `dt_vigencia_ini` (da parametrização), `dt_vigencia_fim = null` (vigência aberta na empresa), `cd_empresa_conveniada = lcd_empresa`, `fl_tipo_minimo`, `qtd_vida`, `vl_multa`, `cd_tabela` (condicional ao plano), `cd_plano`, `dt_cadastro = trunc(sysdate)`, `cd_operador = user`, `vl_faturamento_minimo = null`, `fl_cobra_comissao`. SACTI 1684652.
4	RN-16.04	**Regras de mínimo contratual**	8 campos parametrizáveis: `fl_tipo_minimo` (tipo de cálculo), `qtd_vida` (quantidade mínima de vidas), `vl_multa` (valor da multa por descumprimento), `vl_faturamento_minimo` (da parametrização, mas inserido como NULL na empresa — ⚠ possível bug), `dt_vigencia_ini/fim` (período de vigência), `fl_cobra_comissao` (se cobra comissão sobre o mínimo), `cd_tabela` (tabela de preço vinculada, condicionada a `cd_plano IS NOT NULL`).
5	RN-16.05	**Breakeven padrão 70%**	`INSERT INTO tb_empresa_breakeven (cd_empresa_conveniada, nu_valor, cd_operador, dt_operacao) VALUES (lcd_empresa, 70, user, sysdate)` (~linha 4599). Valor **70** hardcoded = breakeven padrão de 70%. SACTI 1384554 / Devops 42668. Exception `WHEN OTHERS THEN NULL` ? falha não interrompe o fluxo.
6	RN-16.06	**Breakeven vinculado à empresa**	PK = `cd_empresa_conveniada`. Usado para monitoramento da relação receita/custo. Na refatoração, considerar externalizar o valor 70 como parâmetro configurável (`tb_parametro_breakeven`) para permitir breakeven diferenciado por segmento, região ou porte da empresa.

BC-17: Notificação por E-mail

> **Subdomínio:** Notificação (Generic)
> **Package alvo:** `pk_notificacao_email`
> **Procedures:** `pr_send_mail`, `pr_send_mail_html_rn`
> **Responsabilidade:** Enviar e-mail de efetivação ao vendedor/empresa.

Regras de Negócio

#	Código	Regra	Detalhamento
1	RN-17.01	**Diferenciação Hapvida vs RN Saúde**	Branch em 3 níveis (~linha 4676): (1) `IF wcd_empresa_plano = 7 THEN pr_send_mail_html_rn('naoresponda-rnsaude@sh.srv.br', ...)` ? RN Saúde envia HTML com remetente @sh.srv.br. (2) `ELSIF wcd_empresa_plano != 14 THEN` ? verifica **duas vezes** se não é plano 14: primeiro via `wcd_empresa_plano`, depois via `select cd_empresa_plano from tb_filial f, tb_empresa_conveniada a WHERE a.cd_empresa_conveniada = lpad(lcd_empresa,5,'0') AND f.cd_filial = a.cd_filial` ? `IF l_cd_empresa_plano_v != 14 THEN pr_send_mail('naoresponda@hapvida.com.br', ...)` ? Hapvida envia texto plano. **Plano 14 nunca recebe e-mail.** Corpo do e-mail inclui: código do contrato (`lpad(lcd_empresa, 5, '0')`), razão social, CNPJ, endereço completo, **senha em texto claro** (`substr(to_char(lcd_pessoa),(length(lcd_pessoa) - 5),6)`).
2	RN-17.02	**Execução pós-COMMIT**	O envio de e-mail está localizado **após** o `COMMIT` (~linha 4856). O loop de destinatários →: `for st_mail in (select lower(st_e.ds_endereco_eletronico) as endereco from dual)` ? atualmente envia apenas para o e-mail da empresa. Código antigo (comentado) enviava também para vendedor e concessionária via junção `tb_vendedor_plano ? tb_area_venda → tb_endereco_pessoa` e `tb_endereco_eletronico_pessoa`.
3	RN-17.03	**Falha não causa rollback**	`EXCEPTION WHEN OTHERS THEN NULL` no loop de e-mail (~linha 4760). O contrato já está commitado. Falha de envio é silenciosa — não gera log, pendência ou alerta. ⚠ Na refatoração, capturar falha e publicar `EmailFalhouEvent` para retry assíncrono.
4	RN-17.04	**Destinatários**	Atualmente: apenas `st_e.ds_endereco_eletronico` (e-mail da empresa informado na proposta). Código comentado (~linha 4645) mostra que originalmente incluía: vendedor (`tb_vendedor_plano.cd_pessoa → tb_endereco_pessoa`), concessionária (`tb_area_venda.cd_pessoa → tb_endereco_pessoa`), e e-mails adicionais (`tb_endereco_eletronico_pessoa`). A simplificação foi feita sem REMOÇÃO do código morto.

BC-18: Desconto e PIM

> **Subdomínio:** Estrutura Comercial (Supporting)
> **Package alvo:** `pk_desconto_pim`
> **Procedure:** `pr_desconto_empresa`
> **Responsabilidade:** Aplicar descontos administrativos PIM/ADM.

Regras de Negócio

#	Código	Regra	Detalhamento
1	RN-18.01	**Desconto condicionado à flag ativa**	Guard clause (~linha 4770): `if nvl(fn_registro_sistema('FL_ATIVA_PIM_ADM'), 'N') = 'S' then`. A flag `FL_ATIVA_PIM_ADM` ? um Feature Toggle armazenado em `tb_registro_sistema`. Se `'N'` ou NULL ? desconto PIM não é aplicado. Permite habilitar/desabilitar o recurso em produção sem deploy. SACTI xxxxx (Miguel Marcelo).
2	RN-18.02	**Execução via procedure dedicada**	Chamada: `pr_desconto_empresa(pcontrole => st_e.nu_controle, pcd_empresa => lcd_empresa, pcontrol_neg => lnu_controle)` (~linha 4772). Três parâmetros: nu_controle (internet), cd_empresa (definitivo), nu_controle_neg (negociação). A procedure encapsula toda lógica de cálculo e aplicação de desconto PIM/ADM — detalhes internos não visíveis neste código. `EXCEPTION WHEN OTHERS THEN NULL` — falha é silenciosa.

#	Código	Regra	Detalhamento
3	RN-18.03	**Execução pós-COMMIT**	O desconto PIM é executado **após** o `COMMIT` (~linha 4856). Localizado no bloco pós-transacional junto com `pr_cobranca_falta_param` (~linha 4781): `pr_cobranca_falta_param(pcd_empresa_conveniada => lcd_empresa, pnu_controle => lnu_controle, pnu_controle_internet => null)` — verifica parâmetros de cobrança faltantes. Ambas as procedures pós-COMMIT são protegidas por `WHEN OTHERS THEN NULL`. Na refatoração, tratar como Domain Events assíncronos: `ContratoEfetivadoEvent` — handlers para desconto e cobrança.

Regras Transversais (Cross-Cutting)

Estas regras atravessam múltiplos bounded contexts:

#	Código	Regra	BCs Impactados	Detalhamento
1	RN-CC.01	**Contratos AFFIX possuem regras especiais**	BC-04, BC-08, BC-09, BC-12	Guard: `fn_checa_contrato_affix(p_nu_controle, 'S') = 'S'` e `wcd_modelo_negocio != 1`. Impactos concretos: **BC-04**: reutiliza `lcd_pessoa` da empresa de cobrança para endereço (não cria novo). **BC-08**: insere em `tb_empresa_cnpj_contratante` com `nu_cgc_cpf` da proposta (SACTI 449802). `fl_empresa_nova = 'N'`. **BC-09**: `tb_controle_fator_empresa` INSERT com `FL_FATOR_PADRAO` (caminho AFFIX vs caminho normal têm 16 colunas diferentes). **BC-12**: cópia de `tb_emp_limite_acesso_contra` da empresa de cobrança (SACTI 868454). Na refatoração, usar Strategy Pattern `IModeloNegocioStrategy` com implementações `AfixModeloNegocio` e `StandardModeloNegocio`.
2	RN-CC.02	**SIGO vs BITIX como Strategy Pattern**	BC-06, BC-08	Dois cursores paralelos: `cr_empresa_neg` (SIGO: `cd_operados != 'BITIX'`) e `cr_empresa_neg_bitix` (BITIX: `cd_operados = 'BITIX'`). Diferenças concretas: **Resolução de coligada**: SIGO usa `sum(qt_empregados)` agrupado, BITIX usa sum com filtro `cd_operados = 'BITIX'`. **Datas BITIX**: `IF dt_inicio <= sysdate THEN dt_inicio := trunc(sysdate)` (ajuste para não retroagir). **Processamento BITIX**: `V_COUNT_DT_ASSINATURA > 0` habilita UPDATE de titulares/dependentes com `dt_inicio = V_DT_PROCESSAMENTO`. **Canal de venda**: BITIX sempre `wcd_canal_venda := 1`. Na refatoração, extrair `IOrigemPropostaStrategy` com `SigoStrategy` e `BitixStrategy`.
3	RN-CC.03	**Transação única no legado**	Todos	Um único `COMMIT` (~linha 4856) para todas as operações: ~60 INSERTs/UPDATEs em ~47 tabelas. Exception handler global (~linha 4926) faz `ROLLBACK` + INSERT em `tb_pendencia_empresa_internet` + `COMMIT` do log. Blocos pós-COMMIT (BC-13, BC-17, BC-18) com `WHEN OTHERS THEN NULL` são autônomos. Na migração para microserviços: Saga Pattern orquestrado com compensação ? cada BC ? um passo do Saga com evento de compensação correspondente.
4	RN-CC.04	**Log odonto condicional**	BC-14, BC-13	`if nvl(wnu_controle_odonto, 0) > 0 then` aparece 30+ vezes antes de cada INSERT de log. `wnu_controle_odonto` é setado no início da procedure via `select nu_controle from tb_odon_empresa_internet where nu_controle_saude = p_nu_controle`. Dependência circular: o log de erro de saúde depende de existir controle odonto, mas o espelhamento odonto (BC-13) é executado pós-COMMIT. Na refatoração, remover a condição — todo erro deve gerar log independente do contexto odontológico.
5	RN-CC.05	**Naturezas 6 e 9 (PME/Simples) possuem regras especiais**	BC-08, BC-11	**BC-08**: INSERT em `tb_hist_empresa_conveniada` com `cd_assunto = 130` apenas para `cd_natureza IN (6, 9)` (~linha 3710). Valores hardcoded: `ds_hist_empresa = 'HISTORICO DE IMPLANTACAO'`, `dt_historico = dt_inicio`, `cd_operador = user`. **BC-11**: `tb_fidelizacao_empresa` com `dt_fim_fidelizacao = dt_inicio + 1000` (~2,7 anos). Naturezas 6/9 correspondem a PME e Simples Nacional respectivamente, que possuem tratamento regulatório diferenciado pela ANS. Na refatoração, criar `NaturezaJuridicaPolicy` que encapsula as regras específicas por natureza.

Resumo Quantitativo

Métrica	Valor
Bounded Contexts	18
Regras de Negócio (total)	152
Regras Core Domain	66 (BC-01, BC-06, BC-07, BC-08, BC-09, BC-10)

Métrica	Valor
Regras Supporting	66 (BC-02, BC-03, BC-04, BC-05, BC-11, BC-13, BC-15, BC-16, BC-18)
Regras Generic	18 (BC-12, BC-14, BC-17)
Regras Transversais	5
Specifications (validações conceituais)	21 (em BC-02, agrupadas)
Checagens reais no código-fonte	33 (validações desdobradas da camada 4)
Boilerplate de log em validações	~396 linhas (33 × ~12 linhas)
Tabelas de escrita impactadas	47
Tabelas de leitura referenciadas	37+

Rastreabilidade: Regra -> Package -> User Story

BC	Package Alvo	User Story	Prioridade
BC-01	`pk_cadastramento_empresa`	US-04.01, US-04.02	P5
BC-02	`pk_validacao_proposta`	US-01.03	P0
BC-03	`pk_pessoa_juridica`	US-02.02	P1
BC-04	`pk_endereco_comunicacao`	US-02.03	P1
BC-05	`pk_filial_area_venda`	US-02.01	P1
BC-06	`pk_modelo_negocio`	US-03.01	P2
BC-07	`pk_precificacao`	US-03.02	P2
BC-08	`pk_empresa_conveniada`	US-03.03	P3
BC-09	`pk_coparticipacao`	US-03.04	P3
BC-10	`pk_carencia`	US-03.05	P3
BC-11	`pk_fidelizacao`	US-02.05	P4
BC-12	`pk_acesso_internet`	US-02.04	P4
BC-13	`pk_integracao_odonto`	US-02.10	P4
BC-14	`pk_log_auditoria`	US-01.01	P0
BC-15	`pk_reembolso`	US-02.06	P4
BC-16	`pk_minimo_contratual`	US-02.07	P4
BC-17	`pk_notificacao_email`	US-02.08	P4
BC-18	`pk_desconto_pim`	US-02.09	P4

Referências

Documento	Descrição
`ddd-modelagem-dominio.md`	Modelagem DDD completa (Aggregates, Entities, VOs, Services, Events)
`context-map-cadastramento-empresa.cml`	Context Map DSL (Context Mapper)
`README-refatoracao.md`	Roadmap de refatoração (fases e packages)
`BACKLOG-EPICO-FEATURES-USERSTORIES-PLSQL.md`	Backlog completo com User Stories e critérios de aceite
`ESTRATEGIA-REFATORACAO-PLSQL.md`	Estratégia de refatoração PL/SQL → Strangler Fig, 18 packages, `fn_montar_contexto`, roadmap de extração, testes utPLSQL, regras de tradução PL/SQL→C#

Documento gerado em: 2026-03-11
Procedure analisada: `humaster.pr_cadastramento_empresa_prov` (~5.000 linhas PL/SQL)
Total: 18 Bounded Contexts, 152 Regras de Negócio, 5 Regras Transversais